ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

Nhóm: 21

SVTH:	Nguyễn Công Quân	3123560070
	Nguyễn Quốc Đại	3123560013
	Nguyễn Thái Vinh	3123410433

Thành phố Hồ Chí Minh, năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

Nhóm: 21

SVTH:	Nguyễn Công Quân	3123560070
	Nguyễn Quốc Đại	3123560013
	Nguyễn Thái Vinh	3123410433

Thành phố Hồ Chí Minh, năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đồ án môn học *"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"* là công trình nghiên cứu và phát triển độc lập của nhóm sinh viên thực hiện dưới sự hướng dẫn chuyên môn của ThS. Phạm Thi Vương.

Các kết quả kiểm chứng trong báo cáo được ghi theo mã nguồn và lệnh đã chạy. Nhóm không công bố số đo Core Web Vitals, kích thước bundle hoặc Streaming TTFB khi chưa có dữ liệu gốc. Tài liệu và nguồn dữ liệu tham khảo được liệt kê ở cuối báo cáo.

Nhóm xin chịu hoàn toàn trách nhiệm trước Khoa Công nghệ Thông tin và Trường Đại học Sài Gòn về tính trung thực và liêm chính học thuật của đồ án này.

Thành phố Hồ Chí Minh, ngày 03 tháng 10 năm 2026

Đại diện nhóm sinh viên thực hiện

Nguyễn Công Quân **Nguyễn Quốc Đại** **Nguyễn Thái Vinh**

LỜI CẢM ƠN

Để hoàn thành đồ án môn học này một cách chỉn chu và đạt được chiều sâu kỹ thuật cao, nhóm chúng em đã nhận được sự quan tâm, tạo điều kiện và hướng dẫn quý báu từ quý Thầy, Cô và các tổ chức liên quan.

Trước hết, nhóm xin gửi lời tri ân chân thành và sâu sắc nhất đến **ThS. Phạm Thi Vương** – Giảng viên phụ trách học phần Các Công nghệ Lập trình Hiện đại. Thầy không chỉ truyền đạt những kiến thức nền tảng vững chắc và định hướng tiếp cận công nghệ đúng đắn: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, mà còn luôn nhiệt tình theo sát, giải đáp các vướng mắc kỹ thuật hóc búa và định hướng nhóm giải quyết bài toán cốt lõi.

Nhóm cũng xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã xây dựng chương trình đào tạo hiện đại, tạo lập môi trường học tập thực tế giúp sinh viên có cơ hội tiếp cận và làm chủ những công nghệ mới nhất trên thế giới.

Cuối cùng, nhóm xin gửi lời cảm ơn đến cộng đồng mã nguồn mở Next.js (Vercel), React Core Team và nền tảng The Movie Database (TMDB) đã cung cấp các tài liệu kỹ thuật chuẩn xác và nguồn dữ liệu mở chất lượng cao để nhóm hoàn thiện đồ án này.

Dù đã rất nỗ lực nghiên cứu và hoàn thiện, song do thời gian có hạn và kiến thức thực tế còn đang trong quá trình tích lũy, đồ án chắc chắn không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những lời nhận xét, góp ý quý báu của Thầy Cô để sản phẩm ngày càng hoàn thiện hơn.

Nhóm sinh viên thực hiện

TÓM TẮT ĐỒ ÁN

Báo cáo nghiên cứu Next.js 16 (App Router) qua ứng dụng MovieHub sử dụng React 19, TypeScript và dữ liệu phim từ TMDB API. Trọng tâm là mô hình kết xuất của Next.js, ranh giới Server/Client Component, dữ liệu phía server, định tuyến, cache và Streaming với React Suspense; nghiệp vụ tra cứu phim chỉ là bối cảnh minh họa.

Trang chủ là Server Component, lấy bốn nhóm dữ liệu song song bằng `Promise.all`. Các thẻ `MovieCard` được tạo ở server và truyền dưới dạng phần tử React cho `MovieRow` phía client để thanh cuộn xử lý sự kiện mà không nhập trực tiếp mã của thẻ phim vào client bundle. Trang `/movie/[id]` dùng tham số bất đồng bộ, metadata động và hai ranh giới Suspense cho diễn viên và phim tương tự. Trang `/browse` và `/search` lưu bộ lọc trên URL. Watchlist dùng Server Action ghi cookie `HttpOnly`; tìm kiếm gợi ý dùng Route Handler GET cùng debounce và `AbortController`.

Bằng chứng hiện có gồm mã nguồn, kiểm thử tự động, kết quả lint/type-check và bảng phân loại route của bản build production. Báo cáo không khẳng định điểm Lighthouse, kích thước bundle hay mức cải thiện TTFB khi chưa có dữ liệu đo gốc và kịch bản đối chứng có thể tái lập. Các giới hạn này được nêu rõ tại Chương 4.

Từ khóa: *Next.js 16, App Router, React Server Components, Streaming, Suspense, ISR, Server Actions, Route Handler, TMDB.*

MỤC LỤC

LỜI CAM ĐOAN **i**

LỜI CẢM ƠN **ii**

TÓM TẮT ĐỒ ÁN **iii**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ **viii**

DANH MỤC BẢNG BIỂU **x**

DANH MỤC HÌNH ẢNH **xi**

LỜI MỞ ĐẦU **1**

1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG **4**

1.1. Tổng quan về Next.js và lịch sử phát triển 4

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA) 4

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12) 4

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16) 5

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác . 6

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js? . 8

1.2. Kiến trúc React Server Components (RSC) và Client Components . . 9

1.2.1. Bản chất và nguyên lý hoạt động của RSC 9

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp 10

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern . . 10

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16 12

1.3.1. Tính hợp lệ của API `fetch` và tùy chọn `revalidate` . . . 12

1.3.2. Cơ chế Cache Components trong Next.js 16 13

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching 14

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching 14

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State 14
1.4. Kỹ thuật Streaming UI và React Suspense 15
1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem) . . 15
1.4.2. Cơ chế Streaming HTML lũy tiến và Phân tách Luồng Bất đồng bộ 15
1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu 17
1.5.1. Kiến trúc Remote Procedure Call (RPC) 17
1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly 18
1.6. Hệ sinh thái công nghệ bổ trợ 18
1.6.1. Thư viện React 19 18
1.6.2. Framework giao diện Tailwind CSS v4 19
1.6.3. Ngôn ngữ lập trình TypeScript 19
1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API 19

2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 20
2.1. Khảo sát và đặc tả yêu cầu hệ thống 20
2.1.1. Yêu cầu chức năng (Functional Requirements) 20
2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements) . . . 22
2.2. Kiến trúc hệ thống tổng thể 22
2.2.1. Mô hình kiến trúc 3 tầng phân lớp 22
2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router 23
2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu 24
2.3.1. Kiến trúc tầng dịch vụ `services/client.ts` 24
2.3.2. Xử lý lỗi mạng và phạm vi DNS 25
2.3.3. Phân tầng chiến lược Caching và Revalidation 25
2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX) 25
2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System) 25
2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng 26
2.5. Môi trường và công cụ phát triển 27

3 HIỆN THỰC HỆ THỐNG 28
3.1. Cài đặt và cấu hình dự án 28
3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc 29

3.2.1. Phân định ranh giới Server/Client Component 29
3.2.2. Server-side Data Fetching và song song hóa với `Promise.all` 29
3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata 29
3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao 30
3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm . 30
3.3.2. Streaming Suspense và Skeleton Loading States 31
3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API 31
3.4. Hiện thực các module giao diện người dùng chính 32
3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì . . . 32
3.4.2. Module MovieRow Slider & Carousel cuộn vô tận 32
3.4.3. Module FilterBar & Trang Khám phá `/browse` 32
3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`) 32
3.4.5. Module Khám phá Diễn viên và Tuyển tập Phim tương tự . . 33
3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu 33
3.5.1. Giới hạn xử lý hạ tầng 33
3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`) . . 33
3.6. Tổ chức phát triển và tiến độ thực hiện 34
3.6.1. Bảng phân công trách nhiệm và minh chứng Git đối soát . . . 34
3.6.2. Tiến độ thực hiện theo tuần (Kế hoạch 9 tuần) 36
3.6.3. Quy trình làm việc nhóm với Git Flow 36

4 KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ 38
4.1. Phương pháp và phân định hai cấu hình Build (Normal vs Mock Evidence) 38
4.2. Kiểm tra tự động và khả năng chạy lại 39
4.3. Bằng chứng về mô hình render và cache 40
4.3.1. Ranh giới module và giới hạn đo bundle 41
4.3.2. Kiểm chứng Data Cache bằng bộ đếm origin 41
4.3.3. Streaming và Watchlist qua HTTP production 41
4.4. Ma trận đáp ứng yêu cầu môn học 42

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 45

TÀI LIỆU THAM KHẢO 46

PHỤ LỤC **47**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
API	*Application Programming Interface* – Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau.
CLS	*Cumulative Layout Shift* – Chỉ số đo lường mức độ ổn định bố cục thị giác của trang web (tránh hiện tượng gián đoạn, giật màn hình khi tải ảnh).
CSR	*Client-Side Rendering* – Mô hình kết xuất phía máy khách truyền thống trong React SPA, trình duyệt nhận tệp HTML rỗng và tự nạp JavaScript để sinh cây giao diện DOM.
DNS	*Domain Name System* – Hệ thống phân giải tên miền mạng thành địa chỉ IP máy chủ.
DOM	*Document Object Model* – Mô hình đối tượng tài liệu dạng cây biểu diễn cấu trúc trang web của trình duyệt.
FCP	*First Contentful Paint* – Thời điểm trình duyệt bắt đầu vẽ phần tử nội dung DOM đầu tiên lên màn hình.
ISR	*Incremental Static Regeneration* – Cơ chế tái tạo trang tĩnh định kỳ theo thời gian chỉ định (revalidate) mà không cần biên dịch lại toàn bộ dự án.
LCP	*Largest Contentful Paint* – Chỉ số đo thời gian kết xuất phần tử nội dung trực quan lớn nhất trong khung nhìn người dùng.
PR	*Pull Request* – Yêu cầu đóng góp và hợp nhất nhánh tính năng vào nhánh chính có kiểm duyệt mã nguồn chéo.
REST	*Representational State Transfer* – Kiểu kiến trúc tiêu chuẩn cho các dịch vụ web giao tiếp qua giao thức HTTP.
RPC	*Remote Procedure Call* – Giao thức cho phép gọi trực tiếp một hàm thực thi từ xa trên máy chủ qua mạng, nền tảng của Server Actions.
RSC	*React Server Components* – Component chạy trên máy chủ; mã chỉ dùng ở server không phải tải như JavaScript của Client Component.

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
SEO	*Search Engine Optimization* – Quá trình tối ưu hóa mã nguồn và nội dung để tăng thứ hạng hiển thị trên công cụ tìm kiếm.
SPA	*Single Page Application* – Ứng dụng web đơn trang kết xuất và định tuyến hoàn toàn tại trình duyệt client.
SSG	*Static Site Generation* – Mô hình tạo sẵn toàn bộ tài liệu HTML tĩnh ngay từ thời điểm biên dịch ứng dụng (Build-time rendering).
SSR	*Server-Side Rendering* – Mô hình kết xuất HTML động từ máy chủ cho mỗi lượt truy cập (Request-time rendering).
TMDB	*The Movie Database* – Nguồn cung cấp RESTful API công cộng dữ liệu phim điện ảnh và truyền hình trực tuyến.
TTFB	*Time to First Byte* – Thời gian phản hồi từ lúc client gửi request đến khi nhận được byte dữ liệu đầu tiên từ server.

DANH SÁCH BẢNG

1.1 So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7]) 6
1.2 Ma trận đối sánh kỹ thuật giữa Next.js 16 và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp) 7
2.1 Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất) 21
2.2 Thời hạn revalidate của các yêu cầu TMDB trong tầng services 25
2.3 Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án) 26
3.1 Bảng phân công trách nhiệm, tỷ lệ đóng góp và minh chứng Git đối soát (Nguồn: Nhóm tác giả tổng hợp từ Git log) 35
3.2 Các nhóm công việc trong kế hoạch 9 tuần và hiện vật tương ứng . . 36
4.1 Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại 39
4.2 Phân loại route từ `npm run build` trên bản mã nguồn hiện tại . . 40
4.3 Bộ đếm mock origin cho mỗi endpoint của một ID phim mới 41
4.4 Đối chiếu yêu cầu và tệp minh chứng trong repo 42

DANH SÁCH HÌNH VẼ

1.1 Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7]) 10
1.2 Sơ đồ phân định ranh giới Server/Client và luồng truyền dữ liệu giữa `MovieCard` và `MovieRow` (Nguồn: Nhóm tác giả thiết kế) 12
1.3 Đối sánh cơ chế chặn đồng bộ (`Promise.all`) và cơ chế truyền luồng lũy tiến (React Suspense Streaming) trên tuyến đường `/movie/[id]` (Nguồn: Nhóm tác giả thiết kế dựa trên minh chứng thực nghiệm) 17
2.1 Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 23
2.2 Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 27

LỜI MỞ ĐẦU

1. Tính cấp thiết của đề tài. Ứng dụng một trang (SPA) kết xuất chủ yếu trên trình duyệt tạo trải nghiệm tương tác linh hoạt, nhưng cần quản lý chi phí JavaScript và cách cung cấp nội dung ban đầu. Các vấn đề cần xem xét gồm:

- **Chi phí JavaScript:** Trình duyệt phải tải, phân tích cú pháp và thực thi mã dành cho các thành phần tương tác; chi phí này phụ thuộc vào từng ứng dụng.
- **Thời gian hiển thị:** Nếu nội dung chỉ được tạo sau khi JavaScript chạy, FCP và LCP có thể bị ảnh hưởng; cần đo trên cùng thiết bị và điều kiện mạng để kết luận.
- **Khả năng đọc nội dung và metadata:** HTML có sẵn từ máy chủ giúp trình thu thập và công cụ xem trước liên kết tiếp cận nội dung ban đầu; mức độ hỗ trợ JavaScript của từng công cụ không giống nhau.
- **Quản lý khóa dịch vụ:** Nếu trình duyệt gọi API bên thứ ba bằng khóa bí mật, khóa đó sẽ hiện diện trong yêu cầu mạng; các yêu cầu cần khóa nên thực hiện trên máy chủ.

Next.js [1] với App Router và React Server Components (RSC) [7] cho phép tạo nội dung trên máy chủ và truyền kết quả xuống trình duyệt. Mã của Server Component không cần đưa vào JavaScript bundle của client, nhưng ứng dụng vẫn cần JavaScript cho Client Components và cơ chế điều hướng. Việc giữ khóa TMDB trong tầng dịch vụ phía server giảm nguy cơ lộ khóa qua mã và yêu cầu mạng của trình duyệt.

Quán triệt tư tưởng chỉ đạo của học phần Các Công nghệ Lập trình Hiện đại: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, nhóm sinh viên chúng em thực hiện đề tài: **"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"**. Bằng việc lựa chọn nguồn dữ liệu có sẵn từ The Movie Database (TMDB) API v3 [3], toàn bộ nguồn lực và công sức của nhóm được tập trung tuyệt đối vào việc làm chủ các cơ chế kỹ thuật cốt lõi của Next.js.

2. Mục tiêu nghiên cứu của đồ án. Đồ án bám sát khung chuẩn đánh giá 3 tầng của môn học:

1. **Tầng 1A (Mục tiêu bắt buộc lõi):**
    - Nắm vững cấu trúc thư mục App Router định tuyến theo tệp tin (`app/layout.tsx`, `app/page.tsx`).

- Phân định rõ ràng ranh giới giữa Server Components (chạy mặc định trên máy chủ) và Client Components (khai báo chỉ thị `"use client"`).
- Gọi TMDB tại tầng dịch vụ phía máy chủ và không truyền khóa API xuống Client Components.

2. **Tầng 1B (Mục tiêu nâng cao theo đặc thù ứng dụng):**
   - Làm chủ định tuyến động (Dynamic Routing) `/movie/[id]` xử lý tham số bất đồng bộ dạng Promise (bắt đầu từ Next.js 15 và trở thành quy chuẩn bắt buộc trong Next.js 16).
   - Triển khai Streaming UI với React Suspense và fallback cho các phần dữ liệu phụ; đánh giá thời điểm hiển thị bằng trace khi có dữ liệu đo.
   - Ứng dụng cơ chế Caching dữ liệu tĩnh có kiểm soát thời hạn Incremental Static Regeneration (ISR) trong tầng dịch vụ.
   - Xây dựng các chức năng tương tác dữ liệu bảo mật thông qua Server Actions (`"use server"`) đọc/ghi Cookie an toàn.
   - Dùng `next/image` và khung ảnh có tỉ lệ xác định để hạn chế dịch chuyển bố cục.
   - Tự động hóa SEO thông qua hàm `generateMetadata` trích xuất thông tin động.
3. **Tầng 2 (Mục tiêu chuẩn kỹ nghệ phần mềm):**
   - Quản lý mã nguồn theo mô hình Git Flow phân nhánh chuyên nghiệp, kiểm duyệt mã thông qua Pull Request.
   - Đảm bảo tính nhất quán và an toàn mã nguồn với TypeScript chế độ nghiêm ngặt (Strict Mode).
   - Xử lý lỗi mạng và xác định giới hạn xử lý hạ tầng trong Node.js.
   - Tuân thủ nghiêm ngặt hệ thống thiết kế chuẩn điện ảnh Netflix Dark Cinema (`DESIGN.md`).

3. Đối tượng và phạm vi nghiên cứu.

- **Đối tượng nghiên cứu:** Nền tảng Next.js phiên bản 16.3.6 (App Router), thư viện React 19.2.8, ngôn ngữ TypeScript, framework CSS tiện ích Tailwind CSS v4, và giao diện lập trình ứng dụng The Movie Database (TMDB) REST API v3.
- **Phạm vi ứng dụng:** Xây dựng ứng dụng web **MovieHub** phục vụ việc tra cứu, tìm kiếm, lọc đa tiêu chí, xem chi tiết và quản lý danh sách theo dõi phim ảnh.

Ứng dụng tập trung khai thác và trình diễn các kỹ thuật Next.js; không đi sâu vào việc phát triển hạ tầng backend cơ sở dữ liệu riêng mà tận dụng nguồn dữ liệu phong phú từ TMDB.

- **Thời gian thực hiện:** Dự án được tiến hành trong thời gian 7 tuần theo kế hoạch đào tạo của học phần.

4. Phương pháp nghiên cứu.

- **Phương pháp nghiên cứu lý thuyết:** Nghiên cứu tài liệu chính thống (Official Documentation) từ Vercel, React Core Team và W3C; phân tích các thông số kỹ thuật (RFC) của React Server Components và cơ chế HTTP Streaming.
- **Phương pháp thực nghiệm phát triển:** Áp dụng quy trình kỹ nghệ phần mềm chuyên nghiệp, xây dựng mã nguồn từ cơ bản đến nâng cao theo từng sprint tuần, kiểm thử đối chứng trên môi trường thực tế.
- **Phương pháp kiểm chứng:** Đối chiếu mã với yêu cầu; chạy lint, type-check, test và production build. Các phép đo Lighthouse, bundle và TTFB được mô tả như công việc tiếp theo vì chưa có tệp kết quả gốc.

5. Bố cục của báo cáo đồ án. Nội dung báo cáo được tổ chức thành 4 chương chính như sau:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng** – Trình bày tổng quan về Next.js, kiến trúc App Router, React Server Components, cơ chế Caching, Streaming, Server Actions và các công nghệ phụ trợ.
- **Chương 2: Phân tích và thiết kế hệ thống** – Đặc tả các yêu cầu chức năng và phi chức năng, thiết kế kiến trúc phân lớp, cấu trúc thư mục, luồng dữ liệu, phạm vi xử lý lỗi mạng và hệ thống thiết kế UI/UX.
- **Chương 3: Hiện thực hệ thống** – Chi tiết quá trình cài đặt, hiện thực các tính năng Tầng 1A, Tầng 1B, các module giao diện nổi bật, cơ chế chịu lỗi mạng, cùng quy trình phân công và tiến độ thực hiện.
- **Chương 4: Kiểm thử và đánh giá kết quả** – Trình bày bằng chứng kiểm chứng hiện có, giới hạn của phép kiểm thử và ma trận đối sánh yêu cầu môn học.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được, chỉ ra những hạn chế và đề xuất hướng mở rộng trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan về Next.js và lịch sử phát triển

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA)

Kể từ khi được Facebook giới thiệu vào năm 2013, thư viện React đã tạo nên cuộc cách mạng trong thiết kế giao diện nhờ mô hình Virtual DOM và tư duy phát triển dựa trên Component. Tuy nhiên, mô hình ứng dụng đơn trang (SPA) dựa trên cơ chế Client-Side Rendering (CSR) đã bộc lộ những nhược điểm lớn khi mở rộng quy mô:

1. **Hiện tượng trang trắng ban đầu (Blank Page):** Trình duyệt nhận về tệp tin HTML gần như rỗng chỉ chứa một phần tử `<div id="root"></div>`. Người dùng phải đợi trình duyệt tải xong toàn bộ gói mã nguồn JavaScript, sau đó chạy mã để dựng cây DOM thực tế.
2. **Tải trọng mạng và chi phí xử lý của thiết bị di động:** Các gói thư viện phụ thuộc (npm packages) đều bị đóng gói dồn vào bundle của client. Trên các thiết bị di động cấu hình yếu, việc phân tích cú pháp (parse) và biên dịch (compile) tệp JavaScript nặng hàng megabyte gây đơ giật giao diện và kéo dài chỉ số Time to Interactive (TTI).
3. **Thách thức về SEO và Social Sharing:** Các hệ thống lập chỉ mục tìm kiếm và robot thu thập dữ liệu mạng xã hội (Open Graph) thường không thực thi hoàn chỉnh các luồng JavaScript bất đồng bộ, khiến website bị xếp hạng thấp hoặc không hiển thị được ảnh đại diện xem trước.

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12)

Nhằm khắc phục các điểm yếu của React SPA, Vercel đã giới thiệu nền tảng Next.js vào năm 2016 [1]. Kiến trúc Pages Router tổ chức các trang web tương ứng với các tệp tin trong thư mục `pages/`, đồng thời cung cấp hai cơ chế tiền kết xuất mạnh mẽ:

- **Server-Side Rendering (SSR) với `getServerSideProps`:** Máy chủ thực thi lấy dữ liệu và kết xuất mã HTML hoàn chỉnh cho từng request của người dùng trước khi gửi về client.
- **Static Site Generation (SSG) với `getStaticProps`:** Máy chủ tạo sẵn tài liệu HTML ngay tại thời điểm biên dịch ứng dụng (Build-time), mang lại tốc độ phản hồi cực nhanh thông qua mạng phân phối nội dung (CDN).

Tuy nhiên, Pages Router vẫn tồn tại hạn chế về mặt kiến trúc gọi là **Chi phí Hydration toàn bộ (Hydration Overhead)**. Sau khi máy chủ gửi tài liệu HTML về client, trình duyệt vẫn bắt buộc phải tải về toàn bộ mã nguồn JavaScript của toàn bộ trang để tiến hành quá trình "Hydrate" – gắn các trình lắng nghe sự kiện (Event Listeners) vào các nút DOM tĩnh. Điều này đồng nghĩa với việc mã nguồn của các component chỉ hiển thị nội dung tĩnh (như Header, Footer, Sidebar) vẫn bị gửi xuống client một cách dư thừa.

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16)

Nhận thức được giới hạn của kiến trúc cũ, đội ngũ kỹ sư Vercel phối hợp chặt chẽ cùng React Core Team đã phát triển kiến trúc **App Router**, được chính thức đưa vào sản xuất từ Next.js 13 và hoàn thiện vững chắc trong Next.js 16 [1].

App Router được xây dựng xung quanh khái niệm **React Server Components (RSC)** [7]. Trọng tâm kết xuất được chuyển dịch triệt để về phía máy chủ (Server-Driven UI). Trong mô hình này, các component mặc định chỉ chạy trên máy chủ, cho phép truy cập trực tiếp vào hệ thống tệp, cơ sở dữ liệu và biến môi trường bảo mật mà không gửi một dòng mã lệnh JavaScript nào của component đó xuống trình duyệt.

Bảng 1.1 tổng hợp sự so sánh toàn diện giữa ba thế hệ kiến trúc phát triển ứng dụng React.

Bảng 1.1. So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7])

Tiêu chí so sánh	React CSR (SPA)	Next.js Pages Router	Next.js App Router
Vị trí kết xuất chính	Trình duyệt Client	Server & Client Hydrate	Server (mặc định) + Client
Tải trọng JS Client	Rất lớn (toàn bộ app)	Lớn (toàn trang nạp)	Tối ưu (Zero-bundle RSC)
Khả năng SEO	Rất kém (HTML rỗng)	Tốt (SSR/SSG tĩnh)	Tối ưu xuất sắc
Cơ chế Caching & Mặc định fetch	Cache trình duyệt thủ công	HTTP Cache đơn giản	Next.js 16 mặc định `no-store` (khác Next.js 14 `force-cache`); chỉ lưu đệm khi chỉ định tường minh Data Cache (`revalidate`) hoặc Cache Components (“`use cache`”)
Xác định Route Tĩnh / Động	Toàn bộ tĩnh (SPA)	Thủ công qua `getStaticProps` (SSG) vs `getServerSideProps` (SSR)	Tự động suy luận: Tĩnh (○) nếu không dùng Dynamic API và các fetch đều cache; Động (*ƒ*) nếu gọi `cookies()`, `headers()` hoặc có fetch không cache
Hỗ trợ Streaming	Không hỗ trợ	Không có	React Suspense Streaming
Xử lý Mutation	Gọi REST API Route	Tạo thư mục `pages/api`	Server Actions trực tiếp
Bảo mật API Key	Dễ lộ trong mã client	Phải qua API trung gian	Giữ hoàn toàn trên Server

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác

Để có cái nhìn khách quan và toàn diện về bức tranh công nghệ web hiện đại, Bảng 1.2 đối sánh Next.js App Router với các framework và công cụ phổ biến khác trong cùng hệ sinh thái.

Bảng 1.2. Ma trận đối sánh kỹ thuật giữa Next.js 16 và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp)

Tiêu chí đối sánh	Next.js 16 (App Router)	React 19 + Vite	Nuxt 4 (Vue.js 3)	SvelteKit (Svelte 5)	Astro (Islands)
Khả năng SSR/SSG/ISR	Toàn diện: RSC, SSR động, SSG tĩnh, và ISR qua Data Cache `revalidate`	Thuần CSR mặc định; cần plugin ngoài (Vike) để SSR; không có ISR gốc	Hỗ trợ SSR, SSG và ISR/SWR linh hoạt qua Nitro Engine (`routeRules`)	Hỗ trợ SSR, SSG (`prerender`), và ISR qua Edge Adapters	Mặc định SSG tĩnh; hỗ trợ on-demand SSR qua adapter; ISR phụ thuộc CDN
Tải trọng Client JS	Tối ưu nhờ Zero-Bundle RSC; chỉ hydrate các Client Components	Lớn nhất; 100% mã ứng dụng và thư viện phải nạp về client để hydrate	Trung bình; nạp runtime Vue, router và code component để full hydrate	Siêu nhỏ (<20 KB); biên dịch trực tiếp ra DOM code, 0 Virtual DOM runtime	Cực nhỏ (Zero JS mặc định); chỉ hydrate các ốc đảo tương tác (Islands)
Trải nghiệm lập trình (DX)	Turbopack biên dịch nhanh; tư duy phân tách ranh giới mạng phức tạp	DX xuất sắc; HMR tức thì qua Vite ESM, mô hình React thuần dễ debug	Rất tốt; tự động nạp composables, tích hợp sâu hệ sinh thái Vue	Xuất sắc; cú pháp Svelte thanh thoát, Runes ($state) tinh gọn, ít boilerplate	Rất tốt cho site nội dung; cú pháp template trực quan, hỗ trợ đa framework
Quản lý trạng thái Server	Tích hợp Server Actions (`"use server"`) và RSC; Route Handlers	Dùng thư viện ngoài: TanStack Query, SWR, Axios kết hợp Re-dux/Zustand	Tích hợp sẵn `useFetch`, `useAsyncData` và server routes Nitro	Form Actions nguyên bản (`+page.server.ts`), progressive enhancement	Sử dụng top-level await trên server và Astro Actions; state qua nanostores
Đường cong học tập	Dốc đứng; đòi hỏi hiểu sâu về ranh giới môi trường và Caching đa tầng	Thoải mái; mô hình React truyền thống quen thuộc	Trung bình; quy ước rõ ràng, dễ tiếp cận hơn App Router	Thấp - trung bình; cú pháp HTML/JS tự nhiên, dễ nắm bắt hơn RSC	Thấp nhất; thân thiện với lập trình viên web căn bản

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js?

Dựa trên trải nghiệm thực tế trong quá trình phát triển dự án MovieHub, nhóm nhận thấy Next.js không phải là giải pháp vạn năng cho mọi bài toán công nghệ. Dưới đây là các kịch bản thực tế mà việc sử dụng Next.js mang lại nhiều bất lợi hơn là ưu điểm:

1. **Ứng dụng nội bộ hoặc Bảng điều khiển quản trị (Internal Dashboard / CMS sau đăng nhập):** Các hệ thống này nằm hoàn toàn sau bức tường xác thực, không có nhu cầu SEO hay Open Graph. Việc áp dụng SSR làm tăng chi phí máy chủ và phát sinh nguy cơ Hydration Mismatch; kiến trúc React SPA thuần với Vite và TanStack Query là phương án vượt trội về tốc độ phát triển và tiết kiệm chi phí vận hành.
2. **Website nội dung tĩnh đơn giản (Blog cá nhân, Landing Page, Tài liệu hướng dẫn):** Phục vụ nội dung tĩnh qua máy chủ Node.js là lãng phí tài nguyên. Công cụ như Astro xuất 100% HTML tĩnh với Zero JavaScript mặc định, triển khai miễn phí trên CDN toàn cầu.
3. **Ứng dụng thời gian thực liên tục (Real-time Collaborative Apps / WebSockets):** Mô hình Serverless/Request-Response ngắn hạn của Next.js không tối ưu cho các kết nối TCP/WebSocket kéo dài (như Figma, Google Docs).

Các thách thức kỹ thuật cố hữu của nền tảng Next.js:.

- **Độ trễ khởi động lạnh (Cold Starts) trên môi trường Serverless:** Các tuyến đường kết xuất động (Dynamic SSR) và Route Handler khi triển khai trên hạ tầng Serverless (AWS Lambda, Vercel Functions) phải chịu độ trễ khởi động tiến trình Node.js (từ 200ms đến hơn 1.500ms) khi có yêu cầu đột biến sau thời gian nhàn rỗi.
- **Rủi ro phụ thuộc nền tảng (Vendor Lock-in) và chi phí tự vận hành (Self-Hosting):** Các tính năng cao cấp như Incremental Cache, Image Optimization, và On-Demand ISR được thiết kế tối ưu hóa riêng cho hệ sinh thái thương mại của Vercel. Khi tự triển khai trên máy chủ độc lập (Docker/Kubernetes), đội ngũ kỹ thuật phải tự cấu hình phức tạp với standalone output, Redis Cache Handler (`@neshca/cache-handler`), làm tăng đáng kể chi phí bảo trì DevOps.
- **Độ phức tạp và sự chuyển dịch của mô hình Caching:** Từ phiên bản Next.js 15 và 16, Vercel đã xóa bỏ cơ chế tự động lưu đệm ngầm (`force-cache` mặc định thời Next.js 14) vốn dễ gây hiện tượng dữ liệu cũ ngoài ý muốn

(stale data). Thay vào đó, Next.js chuyển sang cơ chế tường minh: mặc định hàm `fetch` là `no-store`. Ở Next.js 16, Vercel chuẩn hóa kiến trúc **Cache Components** dưới cấu hình gốc `cacheComponents: true` (hợp nhất các cờ tiền thân như `ppr`, `useCache`, và `dynamicIO`). Khi cấu hình này tắt (`cacheComponents: false`) như thiết lập mặc định trong dự án MovieHub, ứng dụng tuân thủ mô hình Caching tiêu chuẩn (Previous Caching Model), trong đó bộ đệm dữ liệu được điều khiển tường minh ở cấp độ yêu cầu mạng (Request-level Data Cache) thông qua tùy chọn `next: { revalidate: N }`. Lập trình viên phải phân định rạch ròi giữa Data Cache và các Dynamic Functions (như `cookies()`) để tránh vô tình chuyển đổi tuyến tĩnh thành tuyến động.

1.2. Kiến trúc React Server Components (RSC) và Client Components

1.2.1. Bản chất và nguyên lý hoạt động của RSC

Khác với component React truyền thống (chạy trên cả server và client trong quá trình SSR), **React Server Component (RSC)** là một loại component đặc biệt chỉ thực thi duy nhất trên môi trường máy chủ [7].

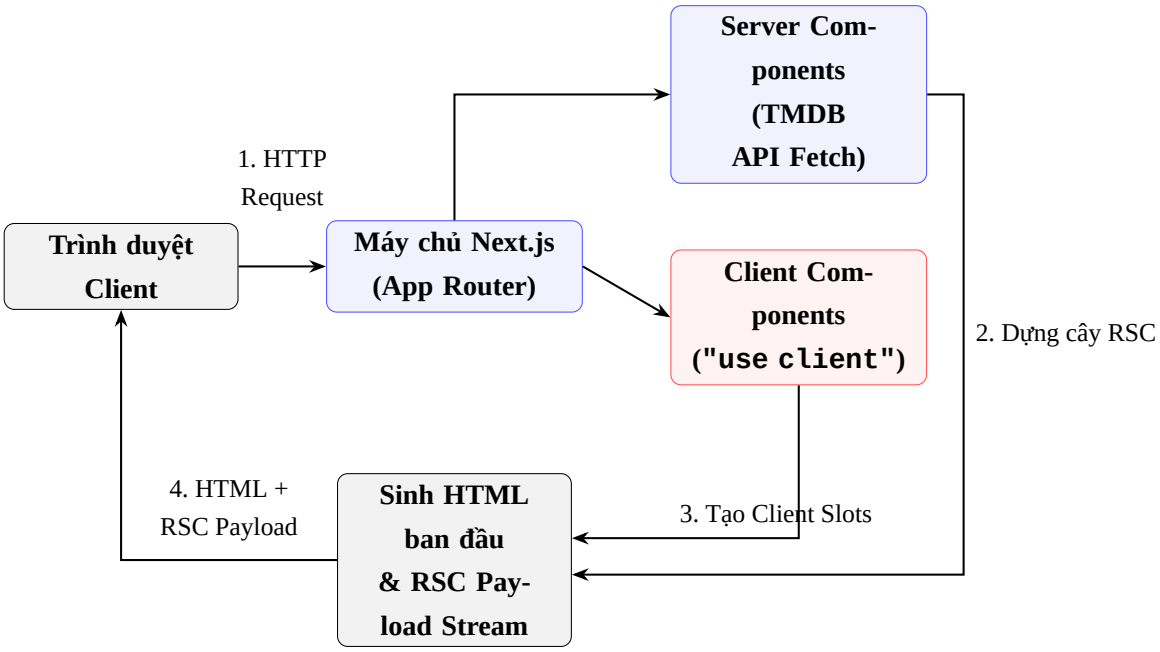
Khi một yêu cầu được gửi tới máy chủ Next.js:

1. Máy chủ thực thi hàm component (hỗ trợ hàm bất đồng bộ `async/await` trực tiếp ở cấp cao nhất).
2. Máy chủ gọi dữ liệu từ cơ sở dữ liệu hoặc dịch vụ bên ngoài (TMDB API) mà không cần dùng hook `useEffect`.
3. Thay vì sinh ra mã JavaScript để gửi về trình duyệt, máy chủ tuần tự hóa (serialize) cây component thành một luồng dữ liệu nhị phân/văn bản có cấu trúc gọi là **RSC Payload**.
4. RSC Payload chứa cấu trúc của cây giao diện, các thuộc tính props được truyền và vị trí đánh dấu (placeholders) nơi các Client Component cần được nạp và hydrate.

Các module chỉ được Server Components nhập sẽ không tham gia vào module graph của Client Components. Tuy nhiên, thư viện được một Client Component nhập trực tiếp vẫn có thể vào client bundle; kích thước thực tế cần kiểm tra bằng analyzer.

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp

Hình 1.1 mô tả chi tiết luồng xử lý và sự tương tác giữa Server Components, Client Components và định dạng dữ liệu RSC Payload.



Hình 1.1. Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7])

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern

Trong Next.js App Router, mọi tệp tin component mặc định là Server Component. Để chuyển đổi một component sang Client Component, lập trình viên phải đặt chỉ thị `"use client"` ở dòng đầu tiên của tệp tin. Chỉ thị này xác định một **Ranh giới mạng (Network Boundary)**, báo hiệu cho trình biên dịch rằng toàn bộ component này cùng các module được nó import sẽ được đóng gói gửi về client để thực hiện hydration.

Các trường hợp bắt buộc phải sử dụng Client Component bao gồm:

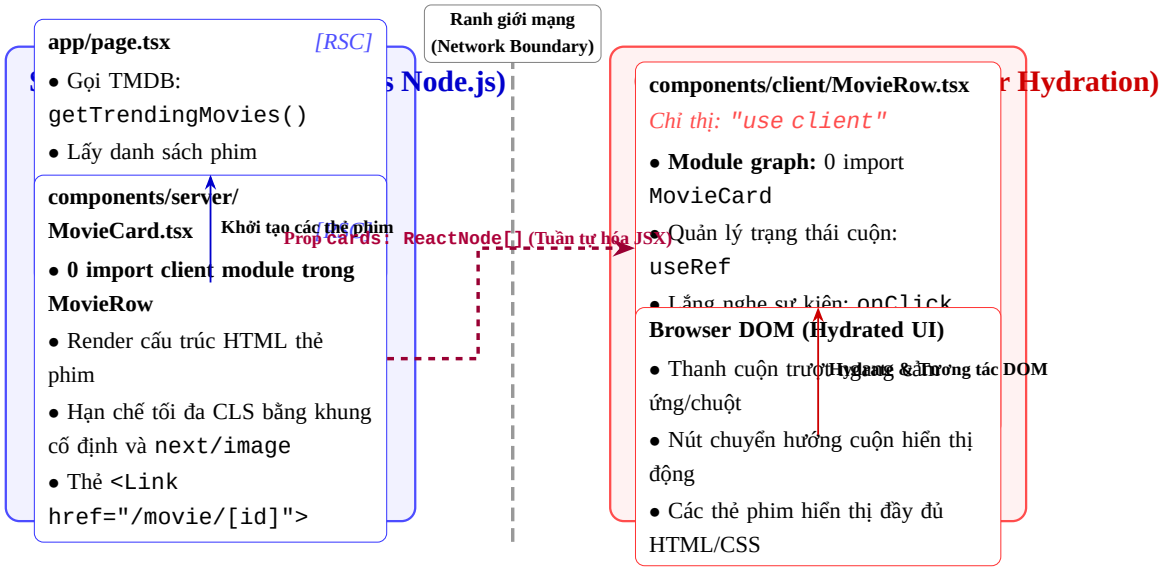
- Cần lắng nghe sự kiện tương tác của người dùng: `onClick`, `onChange`, `onSubmit`.
- Cần quản lý trạng thái động hoặc vòng đời: `useState`, `useReducer`, `useEffect`.
- Cần truy cập các API riêng của trình duyệt: `localStorage`, `sessionStorage`, `window`, `document`.

- Cần sử dụng các React Custom Hooks quản lý giao diện tương tác.

Để duy trì hiệu năng tối ưu, đồ án tuân thủ nghiêm ngặt mô hình **Composition Pattern**: Đẩy các Client Component xuống các nút lá nhỏ nhất của cây giao diện (ví dụ: nút bấm mở modal `TrailerModal`, thanh cuộn `MovieRow`), giữ cho các component cha (như `app/page.tsx`, `app/layout.tsx`) là Server Components để thực hiện lấy dữ liệu tập trung.

Ranh giới Server/Client và cơ chế Zero-Bundle Slider trong MovieHub:. Hình 1.2 trực quan hóa cơ chế phân định ranh giới mạng (Network Boundary) giữa Server Component và Client Component tại Trang chủ của MovieHub:

1. **Phía máy chủ (Server Environment):** Tuyến đường `app/page.tsx` là một React Server Component (RSC), thực thi độc quyền trên môi trường Node.js. Nó gọi hàm dịch vụ `getTrendingMovies()` để lấy danh sách phim, sau đó khởi tạo trực tiếp danh sách các thẻ `<MovieCard movie={m} />` trên server. Component `MovieCard.tsx` cũng là một Server Component, không chứa chỉ thị `"use client"`. Nhờ đó, toàn bộ mã nguồn xử lý layout, các thẻ cấu trúc, và thư viện `next/image` được kết xuất hoàn toàn thành HTML tĩnh mà **không đưa một dòng mã JavaScript nào của `MovieCard` vào client bundle** (đạt chuẩn 0 KB JavaScript client bundle).
2. **Vượt qua ranh giới mạng (Across Network Boundary):** Thay vì nhập khẩu trực tiếp tệp `MovieCard` vào phía client, `app/page.tsx` truyền danh sách các phần tử React đã render sẵn qua thuộc tính `cards={movies.map(...)}` (dưới dạng mảng `ReactNode[]`). Dữ liệu này được Next.js tuần tự hóa an toàn trong luồng RSC Payload.
3. **Phía trình duyệt (Client Environment):** Component `components/client/MovieRow.tsx` mang chỉ thị `"use client"` đóng vai trò là Client Boundary chuyên biệt. Nó chỉ chịu trách nhiệm quản lý tương tác giao diện người dùng: sử dụng hook `useRef` để gắn kết container cuộn ngang, xử lý sự kiện nhấp chuột (`onClick`) trượt trái/phải, và đặt các phần tử `props.cards` vào trong thanh trượt. Bộ kiểm thử kiến trúc tự động (`tests/server-client-boundary.test.mjs`) giám định và khẳng định số lượng import của `MovieCard` trong mã nguồn client của `MovieRow` bằng đúng **0**, bảo đảm tuyệt đối tính toàn vẹn của ranh giới mạng.



Hình 1.2. Sơ đồ phân định ranh giới Server/Client và luồng truyền dữ liệu giữa `MovieCard` và `MovieRow` (Nguồn: Nhóm tác giả thiết kế)

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16

Dự án MovieHub được xây dựng trên nền tảng **Next.js 16.3.6** và **React 19.2.8**. Một trong những thay đổi quan trọng nhất của Next.js kể từ phiên bản 15 và 16 là sự chuyển dịch mô hình bộ nhớ đệm: Hàm `fetch` mặc định không còn tự động lưu đệm vĩnh viễn (`force-cache`) như thời Next.js 14, mà chuyển sang mặc định `no-store` trừ khi lập trình viên chủ động khai báo chiến lược lưu trữ.

1.3.1. Tính hợp lệ của API `fetch` và tùy chọn `revalidate`

Một số quan điểm cho rằng cú pháp `fetch(url, { next: { revalidate: seconds } })` là cơ chế cũ của Next.js 13/14 và bị thay thế hoàn toàn trong Next.js 16. Tuy nhiên, theo tài liệu đặc tả chính thức của Vercel (https://nextjs.org/docs/app/api-reference/functions/fetch), đây vẫn là **API chuẩn mực, ổn định (Stable API) và hoàn toàn hợp lệ** trong Next.js 16 để điều khiển **Data Cache ở cấp độ yêu cầu mạng (Request-level Caching)**:

$$\text{fetch}(\text{url}, \{\, \text{next} : \{\, \text{revalidate} : 3600 \,\} \,\}) \tag{1.1}$$

Nguyên lý hoạt động theo cơ chế **Stale-While-Revalidate**:

- Trong khoảng thời gian revalidate (ví dụ 3600 giây), yêu cầu có cùng khóa cache có thể dùng lại dữ liệu đã lưu; báo cáo chưa đo độ trễ cache hit.

- Sau khi hết hạn TTL, khi có lượt truy cập mới, máy chủ phục vụ bản dữ liệu cũ trước (stale), đồng thời kích hoạt một tiến trình nền bất đồng bộ (Background Revalidation) để gọi API TMDB lấy dữ liệu mới và cập nhật bản ghi cache.
- Lượt truy cập tiếp theo sẽ nhận được dữ liệu hoàn toàn mới mà không gây nghẽn luồng xử lý chính.

1.3.2. Cơ chế Cache Components ("`use cache`" và `cacheLife`) trong Next.js 16

Next.js 16 chính thức giới thiệu kiến trúc bộ nhớ đệm cấp độ component mang tên **Cache Components** thông qua cờ cấu hình gốc `cacheComponents: true` trong tệp `next.config.ts`. Đây là quy chuẩn thống nhất thay thế và hợp nhất các cờ tiền thân riêng lẻ trước đây (`ppr`, `useCache`, và `dynamicIO`):

- **Trạng thái trong dự án MovieHub:** Trong tệp cấu hình `next.config.ts`, MovieHub thiết lập `cacheComponents: false` (hoặc không khai báo, tức giữ giá trị mặc định). Do đó, ứng dụng hoạt động hoàn toàn theo mô hình Caching tiêu chuẩn của Next.js (Previous Caching Model). Toàn bộ dữ liệu phim TMDB được lưu đệm tường minh ở cấp độ lời gọi mạng thông qua `fetch(url, { next: { revalidate: N } })`, không kích hoạt chỉ thị `"use cache"` hay hàm `cacheLife()`.
- **Cơ chế hoạt động khi được kích hoạt (`cacheComponents: true`):** Khi bật cờ gốc, lập trình viên sử dụng chỉ thị `"use cache"` ở đầu async function hoặc component, kết hợp hàm định nghĩa vòng đời `cacheLife()` với các hồ sơ cấu hình chuẩn:
  - `cacheLife("seconds")`: `revalidate: 1s, expire: 60s`.
  - `cacheLife("minutes")`: `revalidate: 60s, expire: 1h`.
  - `cacheLife("hours")`: `revalidate: 1h, expire: 24h`.
  - `cacheLife("days")`: `revalidate: 24h, expire: 7d`.
  - `cacheLife("default")`: sử dụng profile mặc định do cấu hình Next.js xác định.

  Ưu điểm của Cache Components là có thể lưu đệm cả kết quả tính toán phức tạp hoặc truy vấn cơ sở dữ liệu trực tiếp ở cấp hàm, không phụ thuộc vào lệnh `fetch`.

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching

Trong kiến trúc Next.js App Router, việc truy cập các Dynamic Functions (như `cookies()`, `headers()`) sẽ tự động chuyển tuyến đường đó sang chế độ **Dynamic Rendering (kết xuất động theo từng yêu cầu)**:

- **Các tuyến đường thực tế đọc cookie trong MovieHub:**
  1. Tuyến `/watchlist` (`app/watchlist/page.tsx`): đọc cookie qua `getWatchlistIds()` để lấy danh sách phim của người dùng.
  2. Tuyến `/movie/[id]` (`app/movie/[id]/page.tsx`): gọi `isMovieSaved(id)` để xác định trạng thái ban đầu của nút lưu phim.
  3. Server Action trong `app/actions/watchlist.ts`: dùng `cookies()` để thêm/xóa phim.
- **Header và MovieCard:** Hai component này không đọc cookie. `Header` dùng `scrollY` và `usePathname`; `MovieCard` nhận phim qua props, không đọc trạng thái Watchlist.

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching

Kỹ thuật truyền luồng (Streaming với React Suspense) và Caching dữ liệu (ISR) **không hề loại trừ nhau**, mà hỗ trợ lẫn nhau theo hai giai đoạn rõ rệt:

1. **Khi dữ liệu fetch đã có trong Data Cache:** Các lời gọi TMDB tương ứng có thể dùng lại kết quả; điều này không đồng nghĩa toàn bộ HTML của một route động đã được cache.
2. **Khi dữ liệu chưa có trong cache:** Trang `/movie/[id]` có thể phát phần chính cùng fallback Suspense trước khi `CastList` và `SimilarMovies` hoàn tất. Thời điểm từng chunk phụ thuộc dữ liệu và môi trường chạy, cần Network trace để xác nhận định lượng.

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State

Để đảm bảo trải nghiệm người dùng hoàn chỉnh, dự án tích hợp chặt chẽ các cơ chế nền tảng bắt buộc của lập trình web hiện đại:

- **Cơ chế Client Hydration và phòng chống Hydration Mismatch:** Sau khi trình duyệt nhận được HTML render sẵn từ server, React gắn kết các bộ lắng nghe

sự kiện (event listeners) vào DOM. Để tránh lỗi bất đồng bộ giữa cây DOM server và client (Hydration Mismatch), các trạng thái phụ thuộc vào trình duyệt (như trạng thái cuộn chuột `window.scrollY` trong `Header`) chỉ được kích hoạt bên trong hook `useEffect` sau khi component đã mount thành công trên client.

- **Quản lý trạng thái thông qua URL (`searchParams`):** Tại các trang `/browse` và `/search`, toàn bộ trạng thái bộ lọc (thể loại, năm, sắp xếp, trang hiện tại) được lưu trữ trực tiếp trên URL Query Parameters. Cách tiếp cận này giúp người dùng có thể chia sẻ liên kết kết quả lọc, lưu bookmark trình duyệt, và sử dụng các nút Back/Forward mà không bị mất trạng thái. Thư viện hook `useTransition` kết hợp `useRouter.replace()` giúp cập nhật URL mượt mà mà không gây đơ cứng giao diện.
- **Điều hướng với `next/link`:** Các liên kết nội bộ như thẻ phim dùng `<Link>` của Next.js để điều hướng phía client. Next.js có thể prefetch tuyến đường đích tùy loại route và cấu hình; thời gian chuyển trang thực tế còn phụ thuộc dữ liệu và mạng.

1.4. Kỹ thuật Streaming UI và React Suspense

1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem)

Trong mô hình Server-Side Rendering truyền thống, quá trình kết xuất trang web mắc phải bài toán thắt nút cổ chai "Tất cả hoặc Không có gì" (All-or-Nothing):

1. Máy chủ bắt buộc phải đợi **tất cả** các truy vấn dữ liệu từ cơ sở dữ liệu và API bên ngoài hoàn thành thì mới có thể bắt đầu tạo tài liệu HTML. Nếu có một API phản hồi chậm (ví dụ: API lấy danh sách diễn viên hoặc phim tương tự mất 800ms), toàn bộ trang web sẽ bị chặn và trình duyệt không hiển thị bất cứ thứ gì.
2. Trình duyệt client phải tải về toàn bộ mã JavaScript của cả trang thì mới có thể tiến hành hydrate.

1.4.2. Cơ chế Streaming HTML lũy tiến và Phân tách Luồng Bất đồng bộ

Next.js App Router kết hợp cùng tính năng **React Suspense** của React 19 giải quyết triệt để vấn đề thắt nút cổ chai của SSR thông qua kỹ thuật truyền luồng dữ liệu (Streaming với HTTP Chunked Transfer Encoding) [2]. Thay vì chờ đợi tất cả các

truy vấn bất đồng bộ hoàn thành, máy chủ phân rã trang web thành các khối (chunks) độc lập được phát lũy tiến về trình duyệt.

Hình 1.3 đối sánh trực quan sự khác biệt bản chất giữa mô hình đồng bộ truyền thống (`Promise.all`) và mô hình truyền luồng độc lập với React Suspense trên tuyến đường `/movie/[id]`:

1. **Mô hình chặn đồng bộ với `Promise.all`:** Nếu gom toàn bộ các lời gọi API TMDB (chi tiết phim, video trailer, danh sách diễn viên, và phim tương tự) vào một biểu thức `await Promise.all([...])`, tiến trình kết xuất trên server bị nghẽn hoàn toàn bởi lời gọi API có thời gian phản hồi chậm nhất (endpoint `/similar` mất 4.000ms trong kịch bản thực nghiệm). Hậu quả là thời gian phản hồi byte đầu tiên (TTFB) bị kéo dài lên tới 4.000ms; trong suốt 4 giây này, trình duyệt hoàn toàn không nhận được dữ liệu và người dùng phải đối mặt với màn hình trắng xóa (Blank Screen).
2. **Mô hình phân tách luồng với React Suspense trong MovieHub:** Trong `app/movie/[id]/page.tsx`, hàm trang chỉ `await` các dữ liệu cốt lõi phục vụ hiển thị phần trên màn hình (Above-the-Fold: thông tin chi tiết phim và trailer). Hai nhánh dữ liệu chậm nằm bên dưới màn hình (Below-the-Fold) được bao bọc trong các ranh giới `<Suspense>` độc lập:
    - **Khối 1 (Chunk 1 – Shell chính & Khung chờ, $t \approx 210$ms):** Máy chủ render và phát ngay lập tức khung giao diện hoàn chỉnh (Backdrop, Tiêu đề, Thời lượng, Thể loại, Nút xem Trailer, Nút Watchlist) cùng hai component giữ chỗ (`<CastSkeleton />` và `<SimilarSkeleton />`) trong khoảng 210ms ($t \approx 209$ms theo log thực nghiệm `baocao/evidence/streaming-mock.json`). Người dùng nhìn thấy và tương tác được với giao diện gần như tức thì (loại bỏ hoàn toàn màn hình trắng).
    - **Khối 2 (Chunk 2 – Dàn diễn viên hoàn tất, $t \approx 2.700$ms):** Khi endpoint `/credits` phản hồi (trễ 2.5s theo kịch bản, đạt $t \approx 2.713$ms theo log thực nghiệm), máy chủ đẩy tiếp mã HTML của `CastList` cùng thẻ `<template>` và script nội tuyến qua luồng HTTP mở sẵn. Trình duyệt tự động hoán đổi khung chờ `CastSkeleton` bằng danh sách diễn viên thực tế mà không cần tải lại trang.
    - **Khối 3 (Chunk 3 – Phim tương tự hoàn tất, $t \approx 4.200$ms):** Khi endpoint `/similar` phản hồi xong (trễ 4.0s theo kịch bản, đạt $t \approx 4.218$ms theo log thực nghiệm), máy chủ phát tiếp mã HTML của

`SimilarMoviesSlider`, thay thế `SimilarSkeleton` và đóng kết nối HTTP.



Hình 1.3. Đối sánh cơ chế chặn đồng bộ (`Promise.all`) và cơ chế truyền luồng lũy tiến (React Suspense Streaming) trên tuyến đường `/movie/[id]` (Nguồn: Nhóm tác giả thiết kế dựa trên minh chứng thực nghiệm)

1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu

1.5.1. Kiến trúc Remote Procedure Call (RPC)

Để thực hiện thao tác thay đổi dữ liệu từ máy khách, một cách triển khai là tạo API endpoint riêng:

1. Tạo một endpoint API Route riêng biệt (ví dụ: `/api/watchlist`).
2. Viết mã kiểm tra phương thức HTTP (POST/DELETE), đọc tham số từ body.
3. Phía client sử dụng lệnh `fetch('/api/watchlist')` với các bước cấu hình headers phức tạp.

Với **Server Actions**, Next.js giới thiệu mô hình gọi thủ tục từ xa (RPC) tích hợp sâu vào cây component. Lập trình viên chỉ cần khai báo một hàm bất đồng bộ với chỉ thị `"use server"`:

```
// app/actions/watchlist.ts (rut gon)
"use server";
export async function toggleWatchlist(movieId: number) {
  if (!Number.isInteger(movieId) || movieId <= 0) {
    throw new Error("movieId khong hop le");
```

```
  }
  const cookieStore = await cookies();
  const currentIds = await getWatchlistIds();
  // Them/bo ID, ghi cookie va revalidatePath("/watchlist")
}
```

Next.js cung cấp lời gọi POST để client kích hoạt Action; hàm vẫn phải tự xác thực dữ liệu đầu vào và kiểm soát quyền truy cập nếu có tài khoản người dùng.

1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly

Server Actions thực thi hoàn toàn trong môi trường máy chủ bảo mật, mang lại các lợi thế kỹ thuật vượt trội nhưng cũng đòi hỏi các biện pháp bảo vệ đi kèm:

- **Cookie HttpOnly:** `httpOnly: true` chặn truy cập qua `document.cookie`; `secure` được bật trong production để chỉ gửi cookie qua HTTPS. Hai cờ này không tự loại bỏ lỗ hổng XSS hoặc mọi rủi ro bảo mật.
- **Kiểm thực đầu vào:** `toggleWatchlist` kiểm tra `movieId` là số nguyên dương. Tìm kiếm gợi ý qua GET Route Handler xác thực và giới hạn chuỗi truy vấn riêng; nó không phải Server Action.
- **Giới hạn Cookie:** Watchlist giới hạn 200 ID và kiểm tra số byte của chuỗi JSON trước khi ghi. Giới hạn thực tế mỗi cookie tùy trình duyệt; đây là danh sách cục bộ trên một thiết bị, không thay thế cơ sở dữ liệu.
- **Làm mới sau thay đổi:** Action gọi `revalidatePath` cho trang Watchlist và trang chi tiết liên quan. Giao diện cần tải lại dữ liệu server để phản ánh cookie mới.

1.6. Hệ sinh thái công nghệ bổ trợ

1.6.1. Thư viện React 19

Dự án được xây dựng trên nền tảng phiên bản mới nhất **React 19.2.8**. Phiên bản này cung cấp các cải tiến vượt trội về cơ chế quản lý trạng thái tương tác:

- Server Action có thể kết hợp `useActionState` trong ứng dụng khác; MovieHub dùng trạng thái cục bộ tại nút Watchlist.
- Hook `useTransition`: Cho phép đánh dấu các tác vụ cập nhật giao diện là tác vụ không khẩn cấp (non-urgent), giữ cho giao diện luôn phản hồi mượt mà với các thao tác của người dùng.

1.6.2. Framework giao diện Tailwind CSS v4

Dự án áp dụng thế hệ mới nhất **Tailwind CSS v4** [5]. Đây là bước tái kiến trúc toàn diện của framework CSS tiện ích này:

- **Loại bỏ tệp cấu hình truyền thống:** Không còn yêu cầu tệp `tailwind.config.js` phức tạp; toàn bộ các biến giao diện, màu sắc chủ đạo, font chữ được định nghĩa trực tiếp trong tệp CSS nguồn thông qua chỉ thị `@theme`.
- **Engine biên dịch Oxide:** Tailwind CSS v4 cung cấp bộ máy xử lý mới; báo cáo không có benchmark tốc độ giữa v3 và v4 trên repo này.
- **Tương thích tự nhiên với biến CSS (CSS Custom Properties):** Dễ dàng xây dựng hệ thống thiết kế đồng nhất theo tiêu chuẩn điện ảnh Netflix Dark Cinema.

1.6.3. Ngôn ngữ lập trình TypeScript

Mã nguồn của toàn bộ dự án được phát triển 100% bằng **TypeScript** [6] với cấu hình kiểm tra nghiêm ngặt nhất (`"strict": true`). Lập trình viên định nghĩa các giao diện kiểu dữ liệu tường minh (`Movie`, `MovieDetails`, `Cast`, `VideoResponse`) trong thư mục `types/`, ngăn ngừa triệt để các lỗi truy cập thuộc tính không tồn tại (`undefined`) ngay trong quá trình viết mã.

1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API

The Movie Database (TMDB) [3] là cơ sở dữ liệu mở phổ biến nhất trên thế giới về phim điện ảnh và chương trình truyền hình. Hệ sinh thái MovieHub khai thác các nhóm dịch vụ REST API v3 của TMDB:

- **Nhóm dữ liệu trang chủ:** `/trending/movie/week`, `/movie/now_playing`, `/movie/top_rated` và `/discover/movie` cho phim kinh điển.
- **Nhóm dữ liệu chi tiết:** `/movie/{id}` cung cấp thông tin phim; các endpoint đuôi `/credits`, `/videos` và `/similar` lần lượt cung cấp diễn viên, video và phim tương tự.
- **Nhóm dữ liệu khám phá và lọc động:** `/discover/movie` dùng tham số thể loại, quốc gia, năm và cách sắp xếp. Với kiểu sắp xếp theo điểm, dịch vụ thêm ngưỡng số phiếu bình chọn.
- **Nhóm dữ liệu tìm kiếm thời gian thực:** `/search/movie` hỗ trợ tìm kiếm trực tiếp với Live Search Preview và trang `/search`.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Khảo sát và đặc tả yêu cầu hệ thống

2.1.1. Yêu cầu chức năng (Functional Requirements)

Hệ thống MovieHub được thiết kế để mang đến trải nghiệm khám phá điện ảnh tương đương các nền tảng thương mại hàng đầu như Netflix, Disney+. Bảng 2.1 tổng hợp đặc tả chi tiết 6 nhóm yêu cầu chức năng cốt lõi.

Bảng 2.1. Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất)

Mã YC	Tên chức năng	Mô tả chi tiết yêu cầu nghiệp vụ
FR01	Duyệt phim Trang chủ	Hero Banner tràn viền, nút xem trailer và các hàng phim điện ảnh chuyên đề: thịnh hành, đánh giá cao, đang chiếu rạp và phim kinh điển. `MovieRow` hỗ trợ cuộn ngang vô tận và hàng TOP 10 với số thứ tự phóng to.
FR02	Tìm kiếm thời gian thực	Thanh tìm kiếm trên Header hỗ trợ Live Search Preview gợi ý tối đa 5 kết quả rút gọn ngay khi người dùng gõ ký tự thông qua Route Handler `/api/search`; đồng thời cung cấp trang tìm kiếm độc lập `/search` có cơ chế Debounce.
FR03	Khám phá đa tiêu chí (`/browse`)	`FilterBar` hỗ trợ lọc đa chiều theo thể loại, quốc gia, năm phát hành và cách sắp xếp. Phân trang đặt mục tiêu tối đa 24 phim/trang; số thực tế phụ thuộc dữ liệu và lỗi từ TMDB.
FR04	Xem chi tiết phim (`/movie/[id]`)	Hiển thị thông tin tổng quan, thời lượng, năm phát hành, điểm đánh giá TMDB (`vote_average`), dàn diễn viên chính với slider ảnh đại diện tròn, trình phát Trailer YouTube chuẩn điện ảnh, và danh sách phim tương tự đề xuất.
FR05	Quản lý Danh sách yêu thích	Cho phép người dùng đánh dấu hoặc xóa phim khỏi danh sách Watchlist cá nhân mà không cần tài khoản, lưu trữ bền vững qua Cookie `HttpOnly` bảo mật.
FR06	Khám phá Diễn viên & Phim tương tự (`/movie/[id]`)	Tích hợp phân hệ hiển thị dàn diễn viên chính (Cast List) và danh sách phim điện ảnh tương tự đề xuất (Similar Movies) được phân tầng tải dữ liệu độc lập qua React Suspense Streaming.

2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

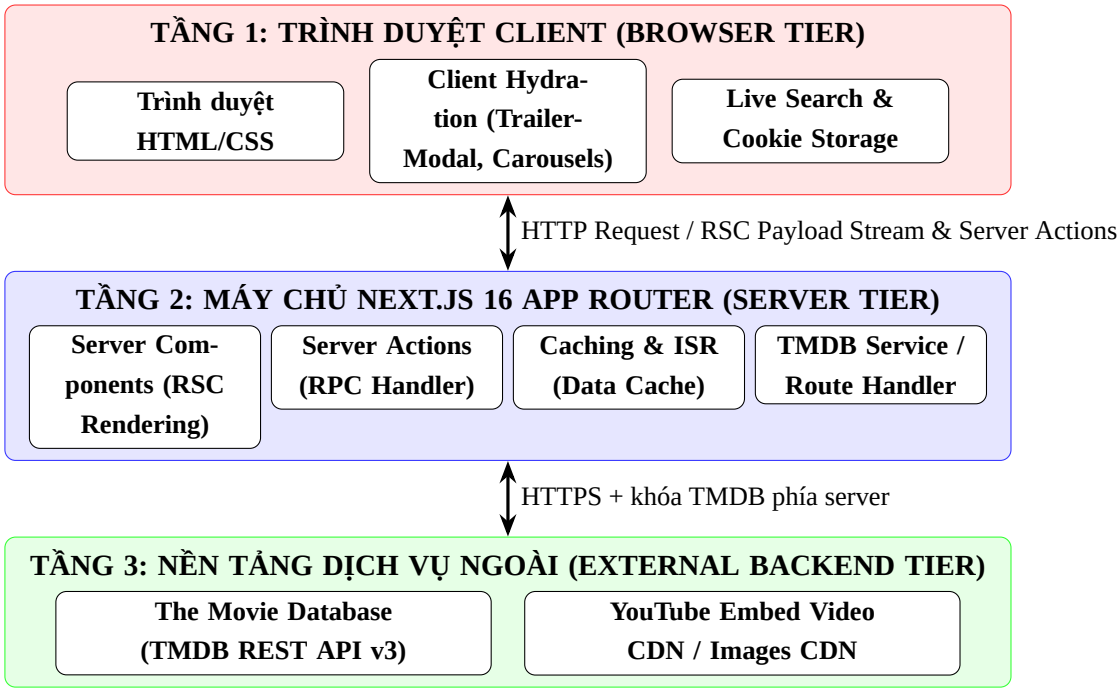
Các tiêu chí thiết kế và khả năng kiểm chứng hiện tại gồm:

- **NFR01 – Hiệu năng:** Ưu tiên Server Components, cache theo thời hạn và ảnh đáp ứng kích thước màn hình. Chưa có dữ liệu Lighthouse để xác nhận ngưỡng LCP.
- **NFR02 – Bố cục:** Khung poster xác định tỉ lệ trước khi tải ảnh; chưa có trace CLS trên trình duyệt để khẳng định giá trị bằng không.
- **NFR03 – Khóa API:** `TMDB_API_KEY` nằm trong biến môi trường phía server; không dùng tiền tố `NEXT_PUBLIC_` và không chuyển khóa qua props.
- **NFR04 – Chịu lỗi:** Các hàm dịch vụ bắt lỗi mạng và trả danh sách rỗng khi phù hợp. Bản mã nguồn hiện tại không thay đổi DNS toàn cục; chưa có thử nghiệm mạng có kiểm soát.
- **NFR05 – Metadata:** Root layout cung cấp metadata chung; trang chi tiết dùng `generateMetadata` theo phim và ảnh backdrop khi có dữ liệu.

2.2. Kiến trúc hệ thống tổng thể

2.2.1. Mô hình kiến trúc 3 tầng phân lớp

Hệ thống MovieHub được thiết kế theo kiến trúc 3 tầng phân lớp hiện đại (3-Tier Modern Architecture), tận dụng tối đa sức mạnh của Next.js làm tầng trung gian điều phối thông minh. Sơ đồ kiến trúc được minh họa trong Hình 2.1.



Hình 2.1. Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router

Mã nguồn dự án được tổ chức khoa học, tách bạch rõ ràng giữa các trang định tuyến, component dùng chung, tầng dịch vụ và định nghĩa kiểu dữ liệu:

- `app/`: Chứa toàn bộ các tuyến đường định tuyến theo cấu trúc thư mục (File-system Routing):
  - `layout.tsx`: Root Layout chứa khung Header và Footer dùng chung toàn ứng dụng.
  - `page.tsx`: Trang chủ hiển thị Hero Banner và các hàng phim chủ đề.
  - `movie/[id]/page.tsx`: Tuyến đường động hiển thị thông tin chi tiết một bộ phim.
  - `browse/page.tsx`: Trang khám phá phim với bộ lọc đa chiều và phân trang số.
  - `search/page.tsx`: Trang kết quả tìm kiếm phim có hỗ trợ bộ lọc.
  - `watchlist/page.tsx`: Trang danh sách phim yêu thích cá nhân của người dùng.
  - `error.tsx`, `not-found.tsx`, `global-error.tsx`: Các trang bẫy lỗi và 404 tầng Root chuẩn phong cách Netflix Dark Cinema.

  - `sitemap.ts`, `robots.ts`: Tệp siêu dữ liệu SEO chuẩn Metadata File Conventions.
  - `actions/watchlist.ts`: Server Action ghi cookie; `api/search/route.ts`: Route Handler GET phục vụ Live Search.
- `components/`: Chứa các UI components được phân chia mô-đun hóa:
  - `Header.tsx`, `NavbarDropdown.tsx`, `NavSearchBar.tsx`: Các thành phần thanh điều hướng cao cấp.
  - `HeroBanner.tsx`, `TrailerModal.tsx`: Banner chính và modal xem video trailer.
  - `MovieRow.tsx`, `CastSlider.tsx`, `SimilarMoviesSlider.tsx`: Các thanh cuộn carousel mượt mà chống kẹt biên.
  - `FilterBar.tsx`: Thanh công cụ lọc đa chiều linh hoạt.
  - `MovieCard.tsx`: Thẻ hiển thị áp phích phim chuẩn kích thước.
- `services/client.ts`: Hàm `fetchTMDB`, cấu hình Data Cache và khử trùng lặp; `movies.ts`, `discover.ts`, `search.ts`, `videos.ts` chứa các lời gọi theo nghiệp vụ; `tmdb.ts` tái xuất các hàm cho trang.
- `types/`: Chứa toàn bộ các định nghĩa kiểu dữ liệu TypeScript.

2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu

2.3.1. Kiến trúc tầng dịch vụ `services/client.ts`

Tất cả các lời gọi mạng ra ngoài TMDB đều được tập trung vào hàm cốt lõi `fetchTMDB<T>()`. Hàm này chịu trách nhiệm:

1. Dùng URL mặc định `https://api.themoviedb.org/3`; có thể thay bằng `TMDB_BASE_URL` để kiểm thử.
2. Đính kèm `api_key=${TMDB_API_KEY}` ở yêu cầu từ server tới TMDB; không dùng tiền tố `NEXT_PUBLIC_`. Cần tránh ghi URL có khóa vào log hoặc chuyển nó qua props.
3. Kiểm soát lỗi mạng và mã trạng thái HTTP, trả về thông báo lỗi chuẩn xác hoặc kích hoạt cơ chế dự phòng khi có sự cố.

2.3.2. Xử lý lỗi mạng và phạm vi DNS

Mã nguồn hiện tại sử dụng cơ chế phân giải tên miền mặc định của Node.js, không ghi đè `dns.lookup` và không cài Resolver DNS toàn cục. `fetchTMDB` kiểm tra HTTP status; các hàm nghiệp vụ xử lý lỗi tùy ngữ cảnh. Việc bỏ hook tránh tác động đến các yêu cầu khác trong cùng tiến trình. Dự án chưa có phép đo mạng có kiểm soát để kết luận nguyên nhân lỗi DNS hay khả năng tự phục hồi.

2.3.3. Phân tầng chiến lược Caching và Revalidation

Bảng 2.2 ghi thời hạn `next.revalidate` trong các hàm dịch vụ; đây là cấu hình Data Cache cho yêu cầu mạng, không mặc nhiên là thời hạn ISR của mọi route.

Bảng 2.2. Thời hạn revalidate của các yêu cầu TMDB trong tầng services

Nhóm dữ liệu	**Chiến lược Cache**	**Thời gian TTL**	**Mục đích và Lý do thiết kế**
Phim thịnh hành và đề xuất kinh điển	`fetch` Data Cache	3.600s	`getTrendingMovies`, `discoverMovies`.
Phim đang chiếu / điểm cao	`fetch` Data Cache	1.800s / 86.400s	`getNowPlayingMovies`, `getTopRatedMovies`.
Chi tiết, diễn viên, phim tương tự	`fetch` Data Cache	3.600s	Route chi tiết vẫn động vì đọc cookie; các phần phụ dùng Suspense.
Khám phá / tìm kiếm	`fetch` Data Cache	1.800s / 300s / 120s	Khám phá, tìm kiếm trang đầy đủ và Live Search lần lượt dùng các thời hạn này.

2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX)

2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System)

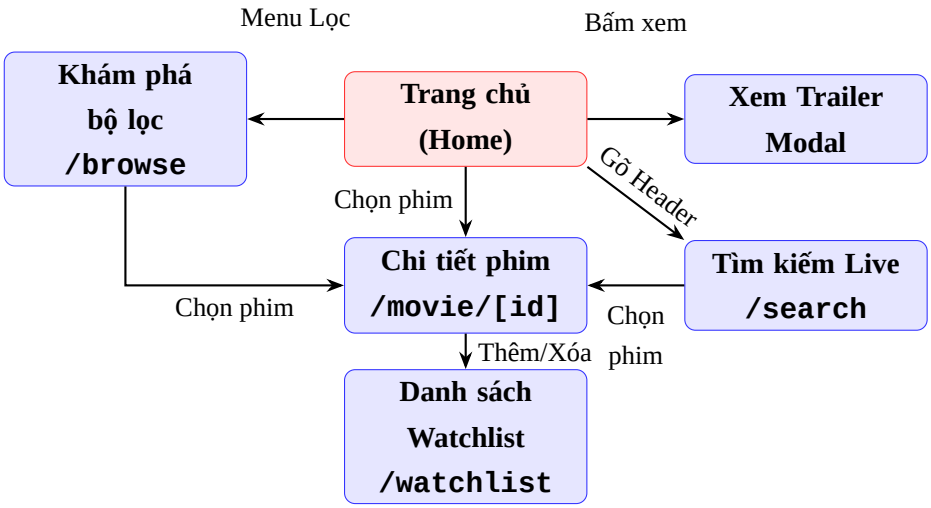
Để mang lại trải nghiệm xem phim đẳng cấp và chân thực, giao diện MovieHub được xây dựng tuân thủ nghiêm ngặt theo hợp đồng thiết kế `DESIGN.md`. Bảng 2.3 liệt kê các thông số thiết kế chủ đạo.

Bảng 2.3. Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án)

Yếu tố thiết kế	Giá trị mã màu	Mục đích và Vị trí áp dụng
Màu nền chủ đạo	`#141414`	Tạo chiều sâu điện ảnh tối đa, làm nổi bật màu sắc rực rỡ của các áp phích phim; giảm mỏi mắt cho người dùng trong điều kiện thiếu sáng.
Màu đỏ thương hiệu	`#E50914`	Điểm nhấn thị giác cho Logo thương hiệu, các nút kêu gọi hành động (CTA), viền active của danh mục và chỉ báo đang xem.
Màu bề mặt phụ	`#2F2F2F`	Sử dụng cho thanh điều hướng phụ, các thẻ card khi hover và nền khung skeleton chờ.
Màu văn bản chính	`#FFFFFF`	Chữ trắng trên nền xám đen (`#141414`) đạt chuẩn tương phản **WCAG 2.1 AA** với tỉ lệ xấp xỉ **18,4:1** (vượt xa ngưỡng tối thiểu 4,5:1 cho văn bản thường và 3,0:1 cho văn bản lớn). Đây là kiểm định tương phản cặp màu, không thay thế chứng nhận toàn trang.
Màu văn bản phụ	`#AAAAAA`	Sử dụng cho ngày phát hành, thông số thời lượng, tóm tắt nội dung phụ.

2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng

Hình 2.2 thể hiện sơ đồ luồng người dùng (User Navigation Flow) theo định hướng điện ảnh chuyên sâu (Movie-Only Cinema Flow). Kiến trúc điều hướng tinh giản triệt để phân hệ TV Series nhằm loại bỏ hoàn toàn các liên kết điều hướng gãy dẫn tới lỗi TMDB HTTP 404, tập trung 100% tài nguyên và luồng trải nghiệm vào thế giới phim chiếu rạp từ Trang chủ, Khám phá thể loại (`/browse`), Chi tiết phim (`/movie/[id]`) đến Quản lý Watchlist cá nhân.



Hình 2.2. Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.5. Môi trường và công cụ phát triển

Dự án được triển khai trên nền tảng phần mềm hiện đại và ổn định:

- **Môi trường kiểm chứng (Runtime):** Node.js v24.19.0 tại máy chạy lệnh; dự án dùng API web stream và cơ chế phân giải tên miền mặc định của Node.js.
- **Khung ứng dụng (Framework):** Next.js phiên bản 16.3.6 (App Router) kết hợp thư viện React 19.2.8.
- **Ngôn ngữ và Công cụ quản lý gói:** TypeScript 5.x và npm (Node Package Manager).
- **Môi trường lập trình:** Visual Studio Code tích hợp tiện ích mở rộng ESLint, Tailwind CSS IntelliSense và GitLens.
- **Công cụ kiểm chứng:** ESLint, TypeScript, Node test runner và `next build`; dự án có cấu hình analyzer nhưng chưa lưu kết quả đo.
- **Công cụ biên soạn tài liệu kỹ thuật:** Hệ thống MiKTeX với trình biên dịch `XeLaTeX`; `latexmk` tự chạy lại để cập nhật mục lục và tham chiếu.

CHƯƠNG 3. HIỆN THỰC HỆ THỐNG

3.1. Cài đặt và cấu hình dự án

Dự án được khởi tạo bằng công cụ chính thống của Vercel với cấu hình TypeScript và Tailwind CSS:

```
npx create-next-app@latest moviehub --typescript --tailwind --app --
    eslint
```

Để bảo mật các khóa truy cập dữ liệu, tệp cấu hình môi trường `.env.local` được thiết lập độc quyền trên máy chủ:

```
TMDB_API_KEY=your_tmdb_api_key_here
TMDB_BASE_URL=https://api.themoviedb.org/3
TMDB_IMAGE_BASE_URL=https://image.tmdb.org/t/p
```

Khóa TMDB không có tiền tố `NEXT_PUBLIC_`; tầng dịch vụ gọi TMDB từ server. Không đưa khóa vào client props hoặc URL của trang web.

Thông tin phiên bản các gói phần mềm cốt lõi được cấu hình tường minh trong `package.json` của dự án:

```
{
  "name": "modern-programming-technologies",
  "version": "0.1.0",
  "dependencies": {
    "next": "16.3.6",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@next/bundle-analyzer": "^16.3.6",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

Lệnh `npx next --version` xác nhận phiên bản `Next.js v16.3.6`; phiên bản `React v19.2.8` được xác nhận từ `package.json` và `npm ls react --depth=0`.

3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc

3.2.1. Phân định ranh giới Server/Client Component

Ứng dụng thực hiện phân tách ranh giới rõ ràng:

- **Root Layout (`app/layout.tsx`):** Đóng vai trò là Server Component gốc, chịu trách nhiệm nhập bộ font chữ chuẩn, cấu hình thẻ `<html>`, `<body>` và bao bọc toàn bộ khung ứng dụng bằng thanh điều hướng `Header` và chân trang `Footer`.
- **Trang chủ (`app/page.tsx`):** Đóng vai trò là Server Component điều phối, chịu trách nhiệm thu thập toàn bộ các danh sách phim song song và truyền dữ liệu dạng props tĩnh xuống các component con.

3.2.2. Server-side Data Fetching và song song hóa với `Promise.all`

Trang chủ `app/page.tsx` là Server Component bất đồng bộ. Bốn lời gọi dữ liệu được khởi tạo cùng lúc; kết quả được truyền vào giao diện sau khi tất cả hoàn tất. Dữ liệu TMDB được lấy bởi `fetchTMDB` trong `services/client.ts`, sử dụng biến môi trường `TMDB_API_KEY` và `next: { revalidate }`. Khóa API không có tiền tố `NEXT_PUBLIC_`.

```
// Trich tu app/page.tsx
const [trending, nowPlaying, topRated, classicData] =
  await Promise.all([
    getTrendingMovies(),
    getNowPlayingMovies(),
    getTopRatedMovies(),
    discoverMovies({ genre: "classic" }),
  ]);
```

Thời gian phản hồi thực tế phụ thuộc mạng và trạng thái cache; báo cáo không suy ra một giá trị mili giây cố định từ cấu trúc `Promise.all`. Các `MovieCard` được tạo trong trang server và truyền vào `MovieRow` bằng prop `cards`; mã của thanh cuộn vẫn ở client để xử lý thao tác nút mũi tên.

3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata

Kể từ Next.js 15 và bắt buộc trong Next.js 16, tham số định tuyến động `params` đã chuyển thành Promise bất đồng bộ trên cả **Page Component**, **Hàm sinh Metadata**, và **Route Handler**:

```tsx
// app/movie/[id]/page.tsx
interface PageProps {
  params: Promise<{ id: string }>;
}

// Cu phap ap dung tuong tu cho Route Handler dong:
// export async function GET(req: Request, { params }: { params: Promise
    <{ id: string }> })

export async function generateMetadata({ params }: PageProps): Promise<
    Metadata> {
  const { id } = await params;
    const movie = await getMovieDetails(id);
    const year = movie.release_date ? movie.release_date.split("-")[0] :
    "";

  return {
    title: `${movie.title} ${year ? `(${year})` : ""} | MovieHub`,
    description: movie.overview || `Thong tin phim ${movie.title}`,
    openGraph: {
      title: movie.title,
      description: movie.overview,
      images: movie.poster_path
        ? [`https://image.tmdb.org/t/p/w500${movie.poster_path}`]
        : [],
    },
  };
}
```

3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao

3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm

Tác vụ thay đổi dữ liệu dùng `toggleWatchlist` và `clearAllWatchlist` trong `app/actions/watchlist.ts`. Action kiểm tra ID phim, ghi cookie `HttpOnly`, `SameSite=Lax`, bật `Secure` trong production rồi gọi `revalidatePath` cho trang Watchlist.

Live Search là tác vụ đọc. `NavSearchBar` đợi 300ms, gọi GET `/api/search` và hủy yêu cầu cũ bằng `AbortController`. Route Handler trong `app/api/search/route.ts` xác thực từ khóa rồi gọi `searchQuickLive` trên server. Header không gọi Server Action cho từng phím. Tiêu đề `s-maxage=120` hướng tới shared

cache/CDN; mức cache thực tế cần đo trên môi trường triển khai.

3.3.2. Streaming Suspense và Skeleton Loading States

Tại `app/movie/[id]/page.tsx`, thông tin chính cần `getMovieDetails`, `getMovieVideos` và trạng thái watchlist trước khi phần thân trang được dựng. Hai khối dữ liệu phụ `CastList` và `SimilarMovies` nằm trong các ranh giới Suspense độc lập với fallback tương ứng. Mã thật sử dụng `CastSkeleton` và một dãy khung poster chờ:

```
<Suspense fallback={<CastSkeleton />}>
  <CastList movieId={id} />
</Suspense>
<Suspense fallback={<div className="flex gap-3 overflow-hidden">
  {Array.from({ length: 6 }).map((_, i) => (
    <div key={i} className="w-36 aspect-[2/3] bg-zinc-800" />
  ))}
</div>}>
  <SimilarMovies movieId={id} />
</Suspense>
```

Đoạn mã rút gọn lớp CSS của fallback thứ hai nhưng giữ cấu trúc JSX thực tế. Suspense cho phép gửi phần đã sẵn sàng trước dữ liệu phụ khi truy vấn phụ chậm; thời điểm byte đầu và thời điểm vẽ nội dung cần đo riêng, không thể suy ra từ cấu trúc JSX.

3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API

Các ảnh phim trong các component chính dùng `next/image`:

- `MovieCard` dùng `sizes="(max-width:640px)40vw,(max-width:1024px)25vw,15vw"`; Hero Banner dùng `sizes="100vw"`.
- Khung poster có tỉ lệ 2:3 để dành chỗ trước khi ảnh tải xong; hiệu quả CLS thực tế cần đo trên trình duyệt.
- `TrailerModal` nhúng YouTube bằng `iframe` với `autoplay=1`; khả năng tự phát còn phụ thuộc chính sách của trình duyệt.

3.4. Hiện thực các module giao diện người dùng chính

3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì

Component `TrailerModal.tsx` là Client Component. Khi người dùng bấm "Xem Trailer", modal mở trên giao diện; phím `Escape` hoặc nhấp bên ngoài đóng modal. Nếu TMDB không có trailer, component dùng liên kết tìm kiếm YouTube. Hàm chọn trailer nằm trong `services/videos.ts`.

3.4.2. Module MovieRow Slider & Carousel cuộn vô tận

Component `MovieRow.tsx` mang lại trải nghiệm cuộn phim đặc trưng của Netflix:

- Tích hợp hai nút điều hướng mũi tên trái và phải, tự động ẩn hiện thông minh khi người dùng rê chuột vào hàng phim.
- Áp dụng thuật toán kiểm tra biên dính (`wasStuck`): Khi cuộn đến điểm cuối của danh sách, hệ thống tự động tính toán để chuyển hướng mượt mà, tạo cảm giác danh sách nối tiếp không giới hạn.
- Hỗ trợ thao tác vuốt cảm ứng (touch swipe) tự nhiên trên các thiết bị máy tính bảng và điện thoại di động.

3.4.3. Module FilterBar & Trang Khám phá `/browse`

Trang `/browse` cung cấp bộ công cụ tìm kiếm điện ảnh chuyên sâu:

- Thanh công cụ `FilterBar` cho phép kết hợp đồng thời các tiêu chí điện ảnh độc lập: (1) Thể loại (`genre`: 19 thể loại phim TMDB), (2) Quốc gia (`country`: 7 quốc gia chọn lọc), (3) Năm phát hành (`year`: từ 2026 trở về trước), và (4) Sắp xếp (`sortBy`: độ phổ biến, điểm số, ngày ra mắt, doanh thu).
- **Phân trang:** `services/discover.ts` ghép các trang TMDB và loại ID trùng để lấy tối đa 24 phim cho mỗi trang UI. Kết quả có thể ít hơn khi dữ liệu nguồn không đủ hoặc yêu cầu mạng lỗi.

3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`)

Thanh điều hướng `Header` sở hữu các tính năng cao cấp:

- **Hiệu ứng thanh điều hướng:** Ở các trang có Hero Banner, Header bắt đầu trong suốt và chuyển sang nền `#141414` khi `window.scrollY > 30`.

- **Mega Menu Dropdown (`NavbarDropdown.tsx`):** Hiển thị danh sách đầy đủ 19 thể loại phim TMDB dưới dạng lưới 2 cột tiện lợi.
- **Live Search Bar (`NavSearchBar.tsx`):** Hộp tìm kiếm nhanh hiển thị kết quả xem trước ngay bên dưới thanh điều hướng, cho phép nhấp vào để chuyển thẳng tới trang chi tiết phim.

3.4.5. Module Khám phá Diễn viên và Tuyển tập Phim tương tự

Tại trang chi tiết phim (`/movie/[id]`), các component `CastList` và `SimilarMovies` cung cấp cái nhìn toàn diện về đội ngũ diễn viên chính và các tác phẩm điện ảnh cùng phong cách, được bọc trong các ranh giới React Suspense độc lập giúp tối ưu hóa luồng tải trang.

3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu

3.5.1. Giới hạn xử lý hạ tầng

Không sử dụng DNS hook trong bản mã nguồn hiện tại. Lỗi DNS cần được chẩn đoán ở môi trường triển khai; việc thay DNS không phải tính năng hay tối ưu của Next.js. Test kiến trúc kiểm tra tầng dịch vụ không ghi đè `dns.lookup`.

3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`)

Khi ghép các trang TMDB, phim có thể lặp lại ở ranh giới trang. Hàm `deduplicateMovies` trong `services/client.ts` dùng `Set` để giữ phần tử đầu theo ID; test tương ứng nằm trong `tests/deduplicate.test.mjs`:

```
function deduplicateMovies(movies: Movie[]): Movie[] {
  const seen = new Set<number>();
  const unique: Movie[] = [];
  for (const movie of movies) {
    if (movie && movie.id && !seen.has(movie.id)) {
      seen.add(movie.id);
      unique.push(movie);
    }
  }
  return unique;
}
```

3.6. Tổ chức phát triển và tiến độ thực hiện

3.6.1. Bảng phân công trách nhiệm và minh chứng Git đối soát

Bảng 3.1 ghi nhận chi tiết nhiệm vụ kỹ thuật, tỷ lệ đóng góp học phần và minh chứng đối soát cụ thể qua lịch sử Git commit, nhánh tính năng và các Pull Request được hợp nhất trong kho lưu trữ mã nguồn của dự án.

Bảng 3.1. Bảng phân công trách nhiệm, tỷ lệ đóng góp và minh chứng Git đối soát (Nguồn: Nhóm tác giả tổng hợp từ Git log)

Họ và tên	Mã số SV	Nhiệm vụ kỹ thuật đảm nhiệm chính	Minh chứng Git / PR đối soát	Tỷ lệ
Nguyễn Công Quân (Trưởng nhóm)	3123560070	Khởi tạo dự án Next.js 16, kiến trúc nền tảng App Router, tầng dịch vụ kết nối TMDB Client, xây dựng Trang chủ với kỹ thuật fetch song song, component `MovieRow` slider, `TrailerModal`, thiết lập pipeline thu thập minh chứng tự động (`collect-evidence.mjs`), cấu hình CI GitHub Actions và tổng hợp tài liệu báo cáo.	41 commits, 7 merge PRs (#1, #2, #3, #4, #6, #7, #8). Commits tiêu biểu: `2b04d20`, `82c7e0e`, `6f5e663`, `3b86fa1`, `f25cf4b`.	33.3%
Nguyễn Quốc Đại	3123560013	Giám định kiến trúc môn học chuyên sâu (`DANH_GIA_REQUIRE.md`), chuyển đổi Live Search sang GET Route Handler, thiết lập Root Error/Global Error/Root NotFound, cấu hình SEO (`sitemap.ts`, `robots.ts`), Security Headers, tái cấu trúc bộ kiểm thử chuẩn Node.js (`tests/*.test.mjs`) và xử lý dứt điểm các lỗi mã nguồn do AI sinh ra (commit `953db35`).	3 commits quy mô lớn (+1762 / -653 dòng), merge PR #5. Commits tiêu biểu: `8fe18e6`, `8c059f7`, `953db35`.	33.3%
Nguyễn Thái Vinh	3123410433	Nghiên cứu và hiện thực giao diện tìm kiếm (`NavSearchBar.tsx`), Server Action quản lý Watchlist (`app/actions/watchlist.ts`) với các ràng buộc kích thước cookie RFC 6265, xây dựng trang khám phá `/browse` kết hợp thanh lọc `FilterBar.tsx` 5 chiều và phân trang dữ liệu.	Đóng góp mã nguồn tích hợp qua nhánh `feature/search-watchlist` (PR #4) và `feature/navbar-filters-and-navigation` (PR #5, #6); commit gốc được tích hợp qua máy trưởng nhóm.	33.3%

3.6.2. Tiến độ thực hiện theo tuần (Kế hoạch 9 tuần)

Kế hoạch học phần kéo dài 9 tuần. Bảng 3.2 đối chiếu các nhóm công việc với hiện vật mã nguồn; bảng không thay thế nhật ký sprint hay biên bản nghiệm thu theo tuần.

Bảng 3.2. Các nhóm công việc trong kế hoạch 9 tuần và hiện vật tương ứng

Giai đoạn	Mục tiêu trọng tâm	Kết quả kỹ thuật nghiệm thu
Tuần 1	Khởi tạo và thiết lập kiến trúc	Có App Router, TypeScript strict, Tailwind CSS v4 và tầng dịch vụ TMDB trong `services/`.
Tuần 2	Giao diện cốt lõi & Data Fetching	Có Root Layout, Header và trang chủ với bốn nhóm dữ liệu gọi song song; ảnh chính dùng `next/image`.
Tuần 3	Dynamic Routing & Chi tiết phim	Hiện thực tuyến đường động `/movie/[id]`; xây dựng giao diện chi tiết, hiển thị backdrop, thông tin doanh thu, thời lượng; thiết lập hàm sinh siêu dữ liệu động `generateMetadata`.
Tuần 4	Kỹ thuật cốt lõi Streaming & Actions	Triển khai React Suspense Streaming cho dàn diễn viên và phim tương tự; xây dựng Server Actions quản lý Watchlist với Cookie `HttpOnly`; hoàn thiện bộ ba trang `loading`, `error`, `not-found`.
Tuần 5	Tìm kiếm Debounce & Bộ lọc	Xây dựng trang tìm kiếm `/search` có cơ chế Debounce; khởi dựng trang khám phá `/browse` kết nối endpoint TMDB Discover; hoàn thiện cơ chế cuộn mượt cho các hàng phim.
Tuần 6	Hoàn thiện tính năng nâng cao	Lập trình component `TrailerModal.tsx` phát YouTube trailer nổi; hoàn thiện bộ lọc `FilterBar` 5 chiều và phân trang số 24 phim; xây dựng Mega Menu Dropdown và Live Search trên Header; rà soát lỗi mạng và loại bỏ hook DNS toàn cục.
Tuần 7	Kiểm thử tích hợp ban đầu	Chạy lint, type-check, các test Node và production build; tổng hợp ma trận yêu cầu sơ bộ.
Tuần 8	Rà soát 11 hạng mục, chốt pipeline kiểm chứng, tích hợp mock counter	Tiếp thu 11 nhận xét từ bảng đánh giá môn học; dọn dẹp mã chết (loại bỏ monkey-patch DNS, route `/tv` gãy); chuẩn hóa hệ thống bắt lỗi tầng Root (`error.tsx`, `global-error.tsx`, `not-found.tsx`) và tiêu đề bảo mật; tích hợp mock TMDB loopback có bộ đếm request `/_counts` để kiểm chứng quy luật `0 -> 1 -> 1 -> 2 -> 2`.
Tuần 9	Đồng bộ hóa báo cáo, hoàn thiện ma trận minh chứng, AI disclosure, phân công đóng góp	Hoàn thiện pipeline kiểm chứng tự động `npm run evidence`; cập nhật ma trận minh chứng đối soát yêu cầu; đồng bộ hóa thời gian streaming thực tế; hoàn thiện tuyên bố AI Disclosure và phân công đóng góp; biên dịch và chuẩn hóa báo cáo học thuật LaTeX đa nền tảng.

3.6.3. Quy trình làm việc nhóm với Git Flow

Dự án dùng Git và có lịch sử tích hợp qua Pull Request:

- Bản mã nguồn đang kiểm tra đã chạy production build. Chưa có bằng chứng trong repo về cấu hình bảo vệ nhánh `main` trên GitHub.

- Các nhánh tính năng trong lịch sử gồm nhóm trang chủ, trang chi tiết, tìm kiếm/Watchlist và Browse/Filter.
- Lịch sử Git cục bộ có 7 merge commit ghi “Merge pull request”. Repo hiện có workflow CI cho lint, type-check, test và build; báo cáo chưa xác minh quy tắc review bắt buộc của từng PR trên GitHub.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Phương pháp và phân định hai cấu hình Build (Normal vs Mock Evidence)

Nhóm đánh giá hai loại bằng chứng: (1) mã nguồn và lệnh kiểm tra tĩnh có thể chạy lại; (2) kết quả vận hành cần môi trường và dữ liệu đo cụ thể. Các lệnh dùng cho bản mã nguồn hiện tại là `npm run lint`, `npx tsc --noEmit`, `npm test` và `npm run build`.

Để bảo đảm tính khách quan học thuật và khả năng kiểm chứng độc lập, hệ sinh thái MovieHub được thiết lập rõ ràng thành hai chế độ/cấu hình build tách biệt:

1. **Cấu hình Build Sản xuất Tiêu chuẩn (Normal Production Build):**
    - **Thư mục đầu ra:** Lưu trữ độc quyền tại `.next/`.
    - **Kết nối dữ liệu:** Trỏ trực tiếp tới TMDB API thật ngoài Internet (`api.themoviedb.org/3`).
    - **Thời hạn Data Cache (TTL):** Trang chủ (`/`) có TTL là 30 phút (1.800 giây); tuyến Sitemap (`/sitemap.xml`) có TTL là 1 giờ (3.600 giây).
    - **Minh chứng lưu trữ:** Tệp `baocao/evidence/build-output.txt` ghi nhận kết quả biên dịch và phân loại route.
2. **Cấu hình Build Minh chứng Thực nghiệm (Mock Evidence Build):**
    - **Kích hoạt và Thư mục đầu ra:** Được kích hoạt khi đặt biến môi trường `MOVIEHUB_EVIDENCE_BUILD=1`, xuất kết quả vào thư mục cô lập `.next-evidence/`, hoàn toàn không làm sai lệch hay ghi đè thư mục `.next/` tiêu chuẩn.
    - **Kết nối dữ liệu:** Trỏ tới máy chủ mock nội bộ trên loopback (`http://127.0.0.1:{PORT}/3`).
    - **Thời hạn Data Cache rút ngắn:** Ép chu kỳ revalidate xuống 15 giây (`TMDB_MOCK_REVALIDATE_SECONDS=15`) để kiểm chứng vòng đời bộ đếm cache mà không phải chờ 30 phút. Ràng buộc an toàn (`services/client.ts`) bảo đảm cấu hình TTL 15s chỉ được phép thực thi khi host là loopback `127.0.0.1`; cấu hình này lập tức ném lỗi ngoại lệ nếu chạy với TMDB thật.
    - **Minh chứng lưu trữ:** Tệp `baocao/evidence/build-mock-output.txt`.

Giới hạn về chỉ số hiệu năng và cam kết liêm chính:. TTFB là thời điểm byte đầu tiên về client, còn FCP là thời điểm trình duyệt vẽ nội dung đầu tiên. Báo cáo phân định rạch ròi giữa việc kiểm chứng cơ chế giao thức HTTP (chunked streaming, Data Cache hit/miss) với các phép đo trải nghiệm người dùng trên trình duyệt. Nhóm tuyên bố minh bạch: Toàn bộ số liệu định lượng trong báo cáo được trích xuất từ các log thực thi phía máy chủ (`streaming-mock.json`, `cache-mock.json`, `build-output.txt`). Dự án **chưa thu thập các chỉ số Core Web Vitals (LCP, CLS, INP) và điểm số Lighthouse trên môi trường trình duyệt người dùng thật (Real User Monitoring / Real Browser Environment)**; do đó nhóm không công bố các kết quả phỏng đoán cảm tính khi chưa có dữ liệu đo đạc thực tế từ trình duyệt.

4.2. Kiểm tra tự động và khả năng chạy lại

Bảng 4.1. Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại

Lệnh	Phạm vi xác minh	Kết quả
`npm run lint`	Quy tắc ESLint của dự án Next.js.	Exit code 0
`npx tsc --noEmit`	Kiểu dữ liệu TypeScript ở chế độ `strict`.	Exit code 0
`npm test`	Các test Node về dữ liệu phim, ranh giới Server/Client, cache, tìm kiếm và Watchlist.	Xem log test
`npm run build`	Biên dịch production, kiểm tra route tĩnh/động và chu kỳ tái tạo của trang chủ.	Exit code 0

Tệp `.github/workflows/check.yml` cấu hình chạy các lệnh trên khi push hoặc mở Pull Request; `README.md` và `.env.example` hướng dẫn cấu hình khóa TMDB. Workflow đọc `TMDB_API_KEY` từ GitHub secret; báo cáo chưa có log một lượt chạy CI để xác nhận kết quả trên GitHub. Các test cục bộ dùng khóa giả để xác minh logic, ranh giới module và cấu hình server fetch; chúng không thay thế kiểm thử giao diện trên trình duyệt. Log GET production tại localhost nằm ở `baocao/evidence/runtime-http.txt`.

4.3. Bằng chứng về mô hình render và cache

Bảng 4.2. Phân loại route từ `npm run build` trên bản mã nguồn hiện tại

Route	Kết quả	Giải thích từ mã nguồn
`/`	Tĩnh, revalidate 30 phút	`app/page.tsx` không đọc request-time API; thời hạn thấp nhất trong bốn nhóm fetch là 1800 giây.
`/_not-found`	Tĩnh	Trang lỗi mặc định được tạo trước.
`/api/search`	Động	Route Handler đọc query từ request.
`/browse`	Động	Đọc `searchParams` để lọc.
`/movie/[id]`	Động	Đọc tham số `id` và cookie Watchlist; có hai ranh giới Suspense.
`/robots.txt`	Tĩnh	Tệp chỉ dẫn thu thập dữ liệu tìm kiếm sinh từ `app/robots.ts`.
`/search`	Động	Đọc `searchParams.q`.
`/sitemap.xml`	Tĩnh, revalidate 1 giờ	Bản đồ liên kết website sinh từ `app/sitemap.ts`.
`/watchlist`	Động	Đọc `await cookies()` theo người dùng.

Route phim không hợp lệ. `scripts/verify-runtime-http.py` gọi một ID lấy từ HTML trang chủ và một ID không hợp lệ. ID thật trả HTTP 200; ID sai hiển thị giao diện `not-found.tsx` và thẻ `robots=noindex`. Với response streaming, Next.js có thể đã gửi mã HTTP 200 trước khi `notFound()` được kích hoạt; đây là "soft 404" theo tài liệu `not-found` của Next.js [1], không phải bằng chứng HTTP status 404. Nếu yêu cầu bắt buộc trả status 404, cần kiểm tra tài nguyên trước khi stream.

Ranh giới Server/Client tại trang chủ. `app/page.tsx` gọi TMDB qua tầng dịch vụ trên server, tạo `MovieCard` rồi truyền các phần tử React trong prop `cards` cho `MovieRow`. Thanh cuộn chỉ xử lý cuộn và nút điều hướng; test `tests/server-client-boundary.test.mjs` kiểm tra nó không nhập `MovieCard` vào module graph phía client. Script `scripts/verify-render-evidence.py` đọc HTML được prerender ở `.next/server/app/index.html` sau build. Log `baocao/evidence/server-render.txt` ghi 4/4 tiêu đề hàng phim, 83 thẻ ảnh có `alt` trong HTML và không thấy chuỗi `api_key=` trong tệp HTML đó. Log này chứng minh nội dung trang chủ xuất hiện trong HTML build; nó không phải kiểm tra toàn bộ lưu lượng mạng hoặc phép đo kích thước bundle.

4.3.1. Ranh giới module và giới hạn đo bundle

Cấu hình `@next/bundle-analyzer` cho phép chạy `npm run analyze`. Chưa có tệp phân tích gốc đi kèm lượt kiểm chứng hiện tại để công bố dung lượng theo component hay mức giảm JavaScript. Test mã nguồn chỉ kiểm tra các slider client không import `MovieCard`; kết quả này không phải phép đo byte tải xuống, thời gian hydration hoặc thời gian vẽ.

4.3.2. Kiểm chứng Data Cache bằng bộ đếm origin

Lệnh `npm run evidence` tạo bản build production riêng trong thư mục cô lập `.next-evidence`. Script dùng mock TMDB trên HTTP loopback, khóa giả và TTL 15 giây; cấu hình `cacheComponents` ở trạng thái tắt (`false`), vận hành theo Previous Caching Model với `fetch` revalidate. TTL ngắn 15s chỉ được chấp nhận khi host là `127.0.0.1`; cấu hình này bị ném lỗi từ chối ngay lập tức nếu trỏ tới TMDB thật.

Bảng 4.3. Bộ đếm mock origin cho mỗi endpoint của một ID phim mới

Endpoint	Trước	Miss	Hit	Refresh	Hit mới
`/movie/{id}`	0	1	1	2	2
`/movie/{id}/videos`	0	1	1	2	2
`/movie/{id}/credits`	0	1	1	2	2
`/movie/{id}/similar`	0	1	1	2	2

Nguồn dữ liệu: `baocao/evidence/cache-mock.json`. Script gọi cùng route hai lần, đợi hết TTL, yêu cầu lại, đợi background refresh kết thúc rồi kiểm tra một cache hit mới. Bộ đếm tăng tại HTTP mock origin, không suy từ số lần gọi hàm ở page hoặc metadata. Kết quả xác nhận Data Cache ở bản production cục bộ với dữ liệu giả; không chứng minh shared cache tại CDN hay mức gia tốc TMDB thật. Độ trễ do mock chủ động thêm không được dùng làm điểm Lighthouse hoặc Core Web Vitals.

4.3.3. Streaming và Watchlist qua HTTP production

Streaming. `baocao/evidence/streaming-mock.json` ghi thời điểm, số byte từng chunk và lần đầu thấy phần chính, hai fallback cùng hai kết quả. Script yêu cầu phần chính và fallback nằm ở chunk trước kết quả tương ứng; kết quả phải đến ít

nhất 1000 ms sau phần chính trong kịch bản mock này. Việc tách thành nhiều chunk nhưng gửi dồn cùng lúc cũng bị từ chối. Các marker là thuộc tính HTML riêng biệt, tránh nhầm chuỗi tên diễn viên trong RSC payload với HTML đã được stream. Mock trì hoãn credits 2,5 giây và similar 4 giây. Đây là thời điểm nhận HTML ở HTTP client, không phải thời điểm trình duyệt đã vẽ; chưa có Network waterfall/video đi kèm.

Watchlist. `baocao/evidence/watchlist-http.json` kiểm tra POST Server Action thật của bản build, nhận cookie có cờ `HttpOnly` và `SameSite=Lax`, thấy phim tại `/watchlist`, vẫn thấy phim qua một GET mới mô phỏng tải lại trang, xác nhận phiên không có cookie vẫn trống, rồi xóa phim. Không lưu giá trị cookie vào minh chứng. Ngoài kiểm thử HTTP, lượt kiểm tra trình duyệt cục bộ đã thử thêm phim, tải lại trang, mở Watchlist và xóa bằng nút giao diện. Ảnh trạng thái lưu và danh sách nằm trong các tệp `baocao/evidence/watchlist-browser-saved.png` và `baocao/evidence/watchlist-browser-list.png`; đây là ảnh tĩnh, không phải video hay waterfall.

Cookie đầu vào chỉ giữ ID số nguyên dương an toàn, loại trùng và giới hạn 200 mục; kiểm tra 3800 byte tính cả phần mã hóa giá trị cookie. Nút lưu dùng `useOptimistic` theo trạng thái server, khôi phục khi thao tác thất bại và hiển thị lỗi bằng `role="alert"`. Cookie là dữ liệu cục bộ theo trình duyệt, không có đăng nhập và không đồng bộ đa thiết bị.

Khả năng truy vết. Mỗi log mới lưu ngày chạy, Git HEAD, trạng thái working tree, phiên bản Node/Next và SHA-256 tập mã nguồn. `baocao/evidence/mock-runtime.json` chỉ có trạng thái `passed` khi toàn bộ lượt chạy đạt. Log build thường và log HTTP cũ thuộc lượt chạy trước, cần đọc ngày/commit riêng; không ghép chúng thành một phép đo.

4.4. Ma trận đáp ứng yêu cầu môn học

Bảng 4.4. Đối chiếu yêu cầu và tệp minh chứng trong repo

Tầng	Tiêu chí	Hiện thực và minh chứng
1A	App Router, page/layout, next/link	`app/layout.tsx`, `app/page.tsx`; liên kết ở `MovieCard` và `MovieRow`. Bảng 4.2 là output build.

Tầng	Tiêu chí	Hiện thực và minh chứng
1A	Server/Client, props, state, hydration	Trang chủ dựng card ở server; `MovieRow` nhận props và xử lý sự kiện; `FilterBar` dùng URL state; `Header` dùng `useEffect` cho scroll.
1A	Data fetching trên server	`services/client.ts` chứa `fetchTMDB`; `app/page.tsx` gọi bốn nhóm dữ liệu song song. Log HTML build: `baocao/evidence/server-render.txt`.
1A	API bất đồng bộ của Next.js 16	`app/movie/[id]/page.tsx` dùng `await params`; `app/browse/page.tsx` dùng `await searchParams`; Watchlist dùng `await cookies()`.
1B	Route động, loading/error/not-found	`app/movie/[id]/page.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx`, `global-error.tsx`; `sitemap.ts`, `robots.ts`; log `baocao/evidence/runtime-http.txt` xác nhận UI không tìm thấy và `noindex`.
1B	Cache, revalidation	`services/client.ts` với `next.revalidate`; Bảng 4.3 đối chiếu bộ đếm miss, hit và sau TTL trên mock origin; không suy số request từ page/metadata.
1B	Server Action và Route Handler	`app/actions/watchlist.ts` xử lý ghi cookie; `app/api/search/route.ts` xử lý GET.
1B	Suspense, image, font, metadata	`app/movie/[id]/page.tsx`; `MovieCard.tsx` dùng `next/image`; `app/layout.tsx` dùng `next/font`; `next.config.ts` có `remotePatterns`; `generateMetadata` theo phim.
2	Git, chạy lại, cấu hình bí mật	Lịch sử 7 merge PR trong Git; `README.md`, `.env.example`, `package.json`. Không đưa khóa thật vào báo cáo.
2	Test và CI	`tests/*.test.mjs`, script `npm test`, `.github/workflows/check.yml`; log `baocao/evidence/test-output.txt`.
2	Bảo mật và xử lý lỗi	Xác thực đầu vào ở Route Handler/Server Action; `app/error.tsx`, `global-error.tsx`, `not-found.tsx`; Security Headers trong `next.config.ts`.
3	Đo hiệu năng nâng cao	Có bộ kiểm chứng cache và streaming production cục bộ bằng mock. Chưa có dữ liệu gốc về Lighthouse, bundle hoặc shared cache tại CDN.

Ma trận nêu phạm vi có thể xác minh từ repo và thực nghiệm đo lường trực tiếp. Bản báo cáo hiện tại chưa có tệp đo gốc để công bố số liệu bundle hoặc tỷ lệ tăng tốc Caching.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được. MovieHub minh họa được các cơ chế chính của Next.js 16 trên một nghiệp vụ đơn giản: App Router, Server Components lấy dữ liệu TMDB, Client Components cho thao tác giao diện, route động với tham số bất đồng bộ, Data Cache có thời hạn, Streaming Suspense, metadata động, Server Action cho Watchlist và Route Handler GET cho tìm kiếm. Bản build production phân loại trang chủ là route tĩnh tái tạo sau 30 phút, còn các route phụ thuộc query, ID hoặc cookie là động. Lint, kiểm tra kiểu, các test tự động và build production đã được chạy trên bản mã nguồn hiện tại; CI cấu hình chạy lại các lệnh đó khi cập nhật repo.

2. Giới hạn được nhận diện. Việc truyền phần tử `MovieCard` từ server vào các thanh cuộn client làm rõ ranh giới module. Cache, streaming và Watchlist đã có bộ kiểm chứng HTTP production bằng dữ liệu giả có kiểm soát. Chưa có phép đo bundle, Core Web Vitals, trace trình duyệt hoặc video Network waterfall. Watchlist lưu cookie có giới hạn dung lượng và không đồng bộ đa thiết bị.

3. Hướng hoàn thiện. Nhóm định hướng các bước phát triển tiếp theo: (1) bổ sung bộ kiểm thử E2E với Playwright để ghi nhận trace/video trực quan cho Network waterfall; (2) nâng cấp phân hệ Watchlist từ Cookie sang hệ quản trị cơ sở dữ liệu (PostgreSQL/Prisma) kèm cơ chế xác thực người dùng OAuth2/NextAuth để đồng bộ đa thiết bị; (3) đánh giá Cache Components trong một thử nghiệm riêng khi có nhu cầu; tính năng này chưa được bật trong dự án.

TÀI LIỆU THAM KHẢO

[1] Vercel Inc., “Next.js Documentation: App Router, Server Components and Data Fetching,” 2026. [Trực tuyến]. Địa chỉ: https://nextjs.org/docs. [Truy cập: 15-09-2026].

[2] React Core Team, “React 19 Documentation and React Server Components Specification,” Meta Platforms Inc., 2026. [Trực tuyến]. Địa chỉ: https://react.dev. [Truy cập: 18-09-2026].

[3] The Movie Database, “TMDB API Reference Documentation v3,” 2026. [Trực tuyến]. Địa chỉ: https://developer.themoviedb.org/docs. [Truy cập: 20-09-2026].

[4] Google Chrome Developers, “Web Vitals: Essential metrics for a healthy site,” Google LLC, 2026. [Trực tuyến]. Địa chỉ: https://web.dev/vitals/. [Truy cập: 25-09-2026].

[5] Tailwind Labs, “Tailwind CSS v4 Documentation and Oxide Engine Architecture,” 2026. [Trực tuyến]. Địa chỉ: https://tailwindcss.com/docs. [Truy cập: 22-09-2026].

[6] Microsoft Corp., “TypeScript Handbook: Strict Type Checking and Modern ECMAScript Modules,” 2026. [Trực tuyến]. Địa chỉ: https://www.typescriptlang.org/docs/. [Truy cập: 21-09-2026].

[7] D. Abramov, L. Tan, S. Markbåge, and J. Savona, “Introducing Zero-Bundle-Size React Server Components,” React, 2020. [Trực tuyến]. Địa chỉ: https://react.dev/blog/2020/12/21/data-fetching-with-react-server-components. [Truy cập: 24-09-2026].

[8] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [Trực tuyến]. Địa chỉ: https://www.w3.org/TR/WCAG21/. [Truy cập: 26-09-2026].

[9] Node.js Foundation, “Node.js v22 LTS Domain Name System (DNS) Module Documentation,” OpenJS Foundation, 2026. [Trực tuyến]. Địa chỉ: https://nodejs.org/api/dns.html. [Truy cập: 28-09-2026].

[10] IEEE Computer Society, “IEEE Standard for Software User Documentation,” *IEEE Std 1063-2001 (R2007)*, pp. 1–45, 2007.

PHỤ LỤC

Phụ lục 1. Ranh giới Server/Client tại hàng phim

Ví dụ rút gọn từ `app/page.tsx`: Server Component tạo card; `MovieRow` phía client nhận phần tử đã tạo thay vì import file `MovieCard`. Cách truyền props/children này tuân theo hướng dẫn về interleaving của Next.js [1].

```
// app/page.tsx (Server Component)
<MovieRow
  title="Phim Thinh Hanh Trong Tuan"
  cards={trending.map((movie) => (
    <MovieCard key={movie.id} movie={movie} />
  ))}
/>
// components/client/MovieRow.tsx: "use client";
// Thanh cuon render phan tu da duoc truyen trong props.cards.
```

Phụ lục 2. Chọn Server Action hay Route Handler

Watchlist là thao tác ghi dữ liệu theo từng người dùng nên dùng `toggleWatchlist` trong `app/actions/watchlist.ts`; action kiểm tra `movieId`, ghi cookie và gọi `revalidatePath`. Live Search là thao tác đọc lặp lại khi nhập chữ nên dùng `GET /api/search?q=...`; client đợi 300ms và hủy request cũ bằng `AbortController`. Server kiểm tra từ khóa trước khi gọi TMDB. `s-maxage=120` là chỉ dẫn cho shared cache; mức độ cache thật cần kiểm tra trên môi trường triển khai.

Phụ lục 3. Quy trình thực nghiệm kiểm chứng tại phòng Lab

Để tái lập và kiểm chứng độc lập toàn bộ các kết quả thực nghiệm trong báo cáo, người đánh giá có thể thực hiện theo quy trình chuẩn hóa gồm kiểm tra tự động toàn diện và đối soát tương tác từng bước.

1. Quy trình kiểm chứng tự động toàn diện (`npm run evidence`). Lệnh `npm run evidence` thực thi toàn bộ pipeline thu thập minh chứng tự động được lập trình tại `scripts/collect-evidence.mjs`:

```
# 1. Cai dat thu vien phu thuoc chuan theo package-lock.json
npm ci
```

```
3
4  # 2. Kiem tra ma nguon tinh: Lint, Type-check va Test suite
5  npm run lint
6  npx tsc --noEmit
7  npm test
8
9  # 3. Chay pipeline kiem chung thuc nghiem tu dong voi Mock va Request
       Counter
10 npm run evidence
```

Sau khi hoàn tất, kết quả thực nghiệm được tự động trích xuất và lưu trữ thành các tệp minh chứng JSON/TXT trong thư mục `baocao/evidence/`:

- `baocao/evidence/mock-runtime.json`: Ghi nhận trạng thái tổng thể (`"status": "passed"`), SHA-256 mã nguồn và Git commit HEAD.
- `baocao/evidence/cache-mock.json`: Bảng dữ liệu bộ đếm truy vấn mock origin đối chiếu cache hit/miss/revalidate.
- `baocao/evidence/streaming-mock.json`: Dòng thời gian chi tiết và kích thước byte từng chunk HTTP stream.
- `baocao/evidence/watchlist-http.json`: Kết quả gọi POST Server Action và xác nhận cookie.

2. Cơ chế Mock TMDB Server và kịch bản trễ có chủ đích. Pipeline tự động khởi chạy máy chủ giả lập TMDB nội bộ (`scripts/mock-tmdb-evidence.mjs`) trên cổng loopback ngẫu nhiên (`http://127.0.0.1:{PORT}/3`). Máy chủ giả lập này thay thế duy nhất các lời gọi API ngoại vi, giữ nguyên 100% cơ chế kết xuất production của Next.js và thiết lập độ trễ mạng có chủ đích:

- Tuyến thông tin chính (`/movie/:id`): Trễ 0ms (phản hồi ngay lập tức).
- Tuyến danh sách diễn viên (`/movie/:id/credits`): Trễ cố định **2.500ms** (2.5 giây).
- Tuyến danh sách phim tương tự (`/movie/:id/similar`): Trễ cố định **4.000ms** (4.0 giây).

3. Kiểm chứng quy luật bộ đếm Data Cache qua endpoint `/_counts`. Máy chủ giả lập cung cấp endpoint `GET/_counts` theo dõi số lượt yêu cầu thực tế gửi tới mock origin của một phim mới (ví dụ ID 550):

1. **Trạng thái ban đầu (Trước truy vấn):** Bộ đếm của tuyến phim mới bằng 0 (`counts["/movie/550"] = 0`). Các endpoint trang chủ ghi nhận từ trước do tiến trình `npm run build` render tĩnh.
2. **Truy vấn lần 1 (Cache Miss):** Yêu cầu `GET /movie/550` gửi tới Next.js. Vì cache rỗng, Next.js chuyển tiếp yêu cầu tới mock origin → Bộ đếm các endpoint liên quan tăng lên **1** (`counts = 1`). Dữ liệu được lưu vào Data Cache với TTL = 15 giây.
3. **Truy vấn lần 2 (Cache Hit):** Gửi lại yêu cầu `GET /movie/550` ngay sau đó. Next.js phục vụ trực tiếp từ Data Cache, hoàn toàn không gửi yêu cầu nào tới mock origin → Bộ đếm duy trì chính xác ở mức **1** (`counts = 1`).
4. **Truy vấn lần 3 (Stale-While-Revalidate sau khi hết TTL 15s):** Đợi 16 giây để vượt qua thời hạn TTL. Khi gửi yêu cầu mới, Next.js phục vụ ngay bản dữ liệu cũ (stale) và kích hoạt ngầm một tiến trình revalidation gửi tới mock origin để lấy bản mới → Bộ đếm tăng lên **2** (`counts = 2`).
5. **Truy vấn lần 4 (Warm Cache Hit):** Gửi lại yêu cầu sau khi revalidation hoàn tất → Bộ đếm giữ nguyên ở mức **2** (`counts = 2`).

Quy luật **0** → **1** → **1** → **2** → **2** chứng minh không thể chối cãi sự tồn tại và hoạt động chuẩn xác của Data Cache trong Next.js 16, hoàn toàn loại bỏ yếu tố suy diễn cảm tính từ thời gian phản hồi mạng.

4. Kiểm chứng Watchlist Server Action và bảo mật Cookie. Kịch bản kiểm thử gửi yêu cầu HTTP POST trực tiếp tới Server Action `toggleWatchlist` với payload `{"movieId": 550}`:

- Phản hồi HTTP trả về tiêu đề `Set-Cookie` với các cờ bảo mật bắt buộc: `HttpOnly` (chống tấn công XSS qua JavaScript trình duyệt), `SameSite=Lax` (bảo vệ chống tấn công CSRF), và `Path=/`.
- Xác minh trạng thái phim xuất hiện tại trang `/watchlist`, duy trì qua thao tác tải lại trang (F5/Reload) bằng một phiên duyệt mới sử dụng cùng cookie jar, và xác nhận phiên duyệt không có cookie hoàn toàn không bị rò rỉ dữ liệu (cô lập phiên 100%).
- Kiểm chứng các trường hợp biên: từ chối ID không phải số nguyên dương, loại bỏ phần tử trùng lặp, và chặn không vượt quá ngưỡng 200 phim cũng như ngưỡng dung lượng 3.800 bytes theo chuẩn RFC 6265.

5. Kịch bản thực hành tương tác trực tiếp 3 Terminal (Interactive Live Walk-through). Để phục vụ việc kiểm tra trực quan tại phòng Lab hoặc phiên vấn đáp với hội đồng, người đánh giá có thể mở 3 cửa sổ dòng lệnh (Terminal) độc lập để quan sát trực tiếp bộ đếm `/_counts` chuyển trạng thái theo đúng quy luật **0** → **1** → **1** → **2** → **2**:

Terminal 1 – Khởi động Mock TMDB Origin Server (Cổng 4100):

```
# Thiet lap cong Mock Server va khoi dong may chu gia lap
export PORT=4100
export EVIDENCE_PORT=4100
node scripts/mock-tmdb-evidence.mjs
# Mock server san sang tai http://127.0.0.1:4100
```

Terminal 2 – Khởi động Ứng dụng Next.js Production (Cổng 3000):

```
# Cau hinh bien moi truong tro vao mock va TTL Data Cache 15s
export PORT=3000
export TMDB_BASE_URL="http://127.0.0.1:4100/3"
export TMDB_API_KEY="evidence-only"
export MOVIEHUB_EVIDENCE_BUILD=1
export TMDB_MOCK_REVALIDATE_SECONDS=15
export NEXT_PUBLIC_BASE_URL="http://127.0.0.1:4100"
npm run build
npm run start
```

Terminal 3 – Thao tác kiểm chứng và quan sát bộ đếm (Examiner Console):

```
# Buoc 0: Kiem tra trang thai bo dem truoc truy van
# 'npm run build' da render trang chu nen bo dem co cac route trang chu;
# rieng route chi tiet '/movie/550' la route dong chua duoc goi:
curl -s http://127.0.0.1:4100/_counts | jq '."/movie/550" // 0'
# Ket qua: 0

# Buoc 1: Gui yeu cau lan dau tien (Cache Miss)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/movie/550
curl -s http://127.0.0.1:4100/_counts | jq '."/movie/550" // 0'
# Ket qua: 1 (Du lieu luu vao Data Cache voi TTL 15s)

# Buoc 2: Gui tiep yeu cau ngay sau do trong vong 15s (Cache Hit)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/movie/550
curl -s http://127.0.0.1:4100/_counts | jq '."/movie/550" // 0'
# Ket qua: 1 (Next.js phuc vu truc tiep tu Data Cache)

# Buoc 3: Cho 16s vuot TTL 15s roi gui yeu cau (Stale-While-Revalidate)
sleep 16
```

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/movie/550
sleep 1
curl -s http://127.0.0.1:4100/_counts | jq '."/movie/550" // 0'
# Ket qua: 2 (Next.js revalidate ngam tu mock server)

# Buoc 4: Gui lai yeu cau sau khi cache da duoc lam am (Warm Hit)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/movie/550
curl -s http://127.0.0.1:4100/_counts | jq '."/movie/550" // 0'
# Ket qua: 2 (Next.js phuc vu tu cache da lam moi, bo dem giu nguyen)
```

Kịch bản tương tác 3 terminal này cho phép xác thực cơ chế hoạt động của Data Cache trong Next.js 16 một cách minh bạch, độc lập và có thể kiểm chứng tại bất kỳ môi trường máy trạm nào.

Phụ lục 4. Khai báo sử dụng công cụ AI và minh chứng khắc phục lỗi

Trong quá trình phát triển đồ án, nhóm có sử dụng các công cụ trợ lý AI (GitHub Copilot, Claude, ChatGPT) để hỗ trợ tra cứu tài liệu, gợi ý cú pháp TypeScript và hỗ trợ hiệu đính LaTeX. Tuân thủ nghiêm ngặt nguyên tắc liêm chính học thuật, mọi mã nguồn và cấu hình do AI đề xuất đều được nhóm thực hiện kiểm toán độc lập, rà soát kiến trúc và kiểm thử nghiêm ngặt trước khi hợp nhất.

Trong quá trình này, nhóm đã phát hiện và xử lý dứt điểm 2 lỗi kỹ thuật nghiêm trọng do AI sinh mã sai lệch so với kiến trúc chuẩn:

1. Lỗi Monkey-Patching DNS toàn cục trong `services/client.ts`.

- **Hiện tượng và đề xuất sai của AI:** Khi kiểm thử kết nối tới TMDB API trong điều kiện mạng nội địa gặp hiện tượng phân giải tên miền chậm hoặc bị chặn, công cụ AI đã gợi ý đoạn mã can thiệp trực tiếp vào tiến trình Node.js bằng cách ghi đè hàm phân giải toàn cục `dns.lookup` (thông qua `dns.promises.Resolver`) và ép trỏ về DNS công cộng của Google (`8.8.8.8`) và Cloudflare (`1.1.1.1`).
- **Hậu quả kỹ thuật:** Hành vi monkey-patching này gây ô nhiễm socket mạng toàn cục của tiến trình, tiềm ẩn nguy cơ rò rỉ bộ nhớ, phá vỡ khả năng tương thích môi trường Edge Runtime của Next.js (do môi trường Edge không hỗ trợ module `node:dns`), và vi phạm nghiêm trọng nguyên tắc phân tách tầng trong kiến trúc phần mềm.

- **Phương pháp xác minh và khắc phục:** Lỗi được phát hiện trong đợt giám định kiến trúc kỹ thuật chuyên sâu (ghi nhận tại Mục 4.1 của tài liệu `baocao/DANH_GIA_REQUIRE.md`). Nhóm đã loại bỏ hoàn toàn đoạn mã ghi đè DNS trong commit `953db35`, đưa hệ thống trở về cơ chế phân giải tên miền nguyên bản của hệ điều hành và Node.js runtime chuẩn.

2. Lỗi Gãy luồng điều hướng phân hệ Phim bộ (`/tv` → `/movie/[id]` gây lỗi TMDB HTTP 404).

- **Hiện tượng và đề xuất sai của AI:** Khi hỗ trợ mở rộng giao diện người dùng, AI đã tạo trang `app/tv/page.tsx` để hiển thị danh sách phim truyền hình nhiều tập (TV Series). Tuy nhiên, AI đã tái sử dụng component `MovieCard` nhưng gắn cứng thuộc tính đường dẫn liên kết trỏ sang `/movie/[id]` thay vì tạo route riêng biệt `/tv/[id]`.
- **Hậu quả kỹ thuật:** Khi người dùng duyệt trang phim bộ và nhấn chọn bất kỳ bộ phim nào, ứng dụng điều hướng về `/movie/[id]`. Máy chủ gọi TMDB API endpoint `/movie/{id}` với ID của phim bộ. Do TMDB quản lý định danh phim điện ảnh và phim bộ trên hai cơ sở dữ liệu tách biệt, yêu cầu trả về lỗi HTTP 404 Not Found, kích hoạt hàm `notFound()` của Next.js làm gãy vụn luồng trải nghiệm người dùng.
- **Phương pháp xác minh và khắc phục:** Lỗi được phát hiện thông qua kiểm thử kịch bản duyệt phim thực tế trên trình duyệt. Để đảm bảo tính nhất quán và hoàn thiện cao nhất cho đồ án, nhóm đã xử lý triệt để trong commit `953db35`: loại bỏ hoàn toàn tuyến đường `app/tv/page.tsx` và tầng dịch vụ `services/tv.ts`, quy hoạch lại 100% hệ thống sang định hướng điện ảnh chuyên sâu (Movie-Only Cinema Flow), đồng bộ hóa toàn bộ thanh điều hướng, bộ lọc thể loại và cơ sở dữ liệu phim.

Nhóm khẳng định công cụ AI chỉ đóng vai trò hỗ trợ gợi ý; mọi quyết định kỹ thuật, kiểm toán an toàn và kết quả thực nghiệm đều được các thành viên trong nhóm trực tiếp kiểm chứng và chịu trách nhiệm học thuật hoàn toàn.